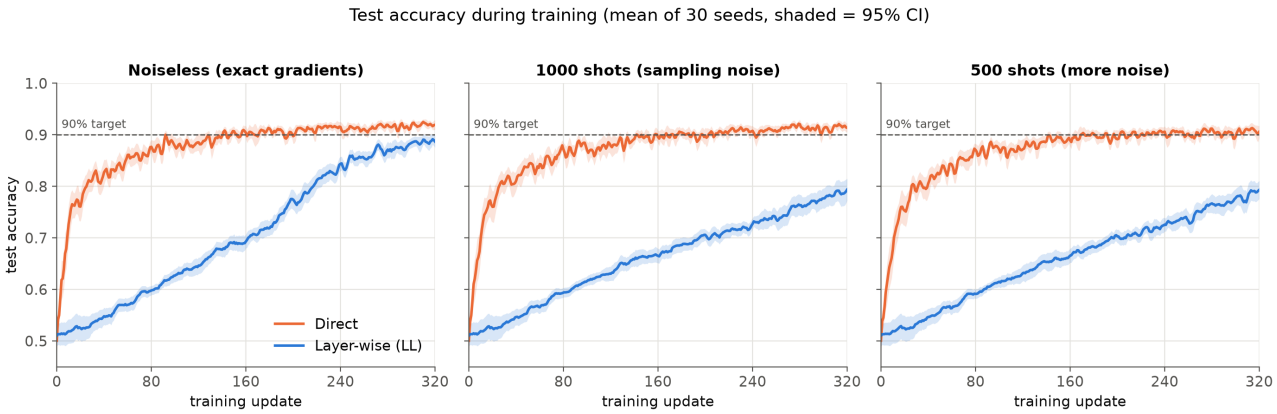
Layer-wise Learning vs Direct Training

of a Variational Quantum Classifier

Mitigating barren plateaus in quantum machine learning:
the complete project guide, explained from zero



Concepts - Data - Quantum circuit - Both training schemes - Every code file
Every graph - Results - Analysis - Viva questions - Glossary

Contents

0. How to use this guide 6

1. The whole project in 2 minutes 7

2. Machine learning basics (classical) 8

2.1 What is classification? . . . 8

2.2 Training, validation and test data . . . 8

2.3 The model outputs a probability . . . 8

2.4 The loss: measuring how wrong we are . . . 8

2.5 Gradient descent: walking downhill . . . 9

2.6 Mini-batches, updates and epochs . . . 9

2.7 Adam: a smarter gradient descent . . . 9

2.8 Random seeds and why we use 30 of them . . . 9

3. Quantum computing basics 11

3.1 Bits and qubits . . . 11

3.2 Measurement and probabilities . . . 11

3.3 Expectation value Z . . . 11

3.4 Quantum gates used in this project . . . 12

3.5 Many qubits and entanglement . . . 12

3.6 How to read a circuit diagram . . . 12

4. Variational quantum circuits (quantum neural networks) 13

4.1 The idea . . . 13

4.2 Layers and depth . . . 13

4.3 How gradients are measured on a quantum computer: the parameter-shift rule . . . 13

5. Barren plateaus 15

5.1 What a barren plateau is . . . 15

5.2 Why it happens (intuition) . . . 15

5.3 Why it is so bad with real hardware . . . 15

5.4 What makes it better or worse . . . 15

5.5 How we measure it in this project . . . 15

6. Layer-wise learning (the proposed fix) 17

6.1 The idea (Skolik et al., 2021) . . . 17

6.2 Why it should help . . . 17

6.3 Two variants . . . 17

7. The problem statement and our objectives 18

8. The task and the data (Objective O1) 19

8.1 MNIST, digits 4 vs 9 . . . 19
8.2 From a picture to 10 angles . . . 19
8.3 The three subsets . . . 20
9. The quantum classifier (our model) 21
9.1 The circuit, part by part . . . 21
9.2 Why these design choices . . . 21
9.3 The light cone (important for the analysis) . . . 22
10. The two training protocols (Objective O2) 23
10.1 Direct training . . . 23
10.2 Layer-wise training (LL) . . . 23
10.3 What makes the comparison fair . . . 23
11. Noise: training with a limited number of shots 25
12. How we measure performance 26
13. The experiment as a whole 27
14. The code: overview 28
14.1 The folder . . . 28
14.2 How the files work together . . . 28
14.3 Python in 1 minute (enough to read the code) . . . 28
15. config.py: all settings 30
16. data.py: preparing MNIST 31
16.1 Loading the data . . . 31
16.2 Shrinking images and choosing balanced subsets . . . 32
16.3 The full preparation pipeline . . . 32
17. circuit.py: the model in PennyLane 34
18. fastsim.py: the fast simulator 36
18.1 What a simulator does . . . 36
18.2 Setting up . . . 36
18.3 Encoding, layers and the output . . . 37
18.4 Parameter-shift values and shot sampling . . . 38
19. train.py: the two training schemes 40
19.1 The schedule and the starting angles . . . 40
19.2 Adam that only updates the active layers . . . 41
19.3 The model and its gradients . . . 42
19.4 The training loop . . . 43
19.5 The summary of one run . . . 44

20. runner.py and run_all.py: running the experiment **46**

20.1 runner.py 46

20.2 run_all.py 47

21. gradients.py: the barren-plateau diagnostic **49**

22. figures.py and the notebook **51**

22.1 figures.py: table and graphs 51

22.2 The notebook 56

23. Results: the numbers (Objective O3) **57**

23.1 The main table 57

23.2 Is the difference real? (significance) 57

23.3 No overfitting 57

23.4 Comparison with classical models 57

24. Every graph explained **59**

24.1 Figure 1: the circuit 59

24.2 Figure 2: the two training schemes 59

24.3 Figure 3: the barren-plateau diagnostic 60

24.4 Figure 4: test accuracy during training 61

24.5 Figure 5: final test accuracy and noise sensitivity 62

24.6 Figure 6: percentage of successful runs 62

24.7 Figure 7: iterations to converge 63

25. A deeper look inside training **65**

25.1 The training loss over time (average of 30 runs) 65

25.2 Gradient size per parameter during training (noiseless) 65

26. Analysis: why Layer-wise learning did not win here **66**

26.1 Reason 1: Direct is not on a barren plateau at 10 qubits 66

26.2 Reason 2: shallow circuits cannot see the whole image (light cone) 66

26.3 Reason 3: smaller gradients suffer more from shot noise 66

26.4 When would layer-wise learning win? 66

27. Conclusion **68**

28. Limitations and future work **69**

29. How we know everything is correct **70**

30. Running the project and demo for sir **71**

30.1 One-time setup (Windows) 71

30.2 Commands 71

30.3 Showing the notebook 71

30.4 Suggested 10-minute presentation order 71

31. Viva questions and answers **72**

31.1 Basics 72

31.2 Our design 72

31.3 Results and analysis 73

32. Glossary **75**

References 76

0. How to use this guide

This guide explains the whole project from zero. You do not need any background in quantum computing or machine learning. Read it in order the first time. Every chapter ends with a green **"Say this to sir"** box: a short version you can say out loud.

Part	Chapters	What you get
A. The story in 2 minutes	1	The whole project in one page; memorise this first
B. Background from zero	2-6	Machine learning, qubits, gates, measurement, variational circuits, barren plateaus, layer-wise learning
C. Our project	7-13	Problem statement, data, circuit, the two training schemes, noise, how we measure success
D. The code, file by file	14-22	Every code file, shown in full with line numbers, explained piece by piece
E. Results and graphs	23-25	Every graph: what it shows, how to read it, what to say
F. Analysis	26-28	Why the results came out this way, conclusion, limitations
G. Proof it is correct	29	All checks we did
H. Running and demo	30	How to run things in front of sir
I. Viva questions	31	About 40 likely questions with simple answers
J. Glossary	32	Every technical word in one line

Colour code used in all graphs: **orange = Direct training**, **blue = Layer-wise training (LL)**.

> **Key idea:** The most important single fact: we built and fairly compared both training schemes. The barren plateau is real (our gradient measurement shows it), and layer-wise learning does avoid it at the start. But on our 10-qubit problem, Direct training was **better**, and we can explain why. An honest negative result explained well is a strong project.

1. The whole project in 2 minutes

The problem. Quantum computers can run small "quantum neural networks" called *variational quantum circuits* (VQCs). They are trained like normal neural networks: a classical computer slowly adjusts the angles of the quantum gates to reduce the error. The danger is the **barren plateau**: for big, deep circuits with random starting angles, the slope (gradient) of the error becomes almost exactly zero everywhere. The optimiser then has no idea which way to move, and training fails.

The proposed fix. Layer-wise learning (LL) (Skolik et al., 2021): start training with a very shallow circuit (1 layer), whose gradients are large, and add one layer at a time during training. Each new layer starts as (almost) "do nothing", so it does not destroy what was learned.

What we did.

- **Task (O1):** classify real handwritten digits **4 vs 9** from MNIST with a 10-qubit, 16-layer quantum circuit.
- **Two schemes (O2):** (a) **Direct**: train all 16 layers from the start; (b) **Layer-wise**: start with 1 layer, add a layer every 20 updates. Both get exactly the same budget: 320 training updates.
- **Comparison (O3):** 30 random seeds per scheme, in 3 noise settings: exact (noiseless), 1000 measurement shots, 500 shots. That is 180 training runs. We measured final accuracy, iterations to converge, success rate and sensitivity to noise.

What we found.

	Direct	Layer-wise
Test accuracy, noiseless	**0.92**	0.89
Test accuracy, 1000 shots	**0.91**	0.79
Test accuracy, 500 shots	**0.91**	0.79
Successful runs (noiseless / 1000 / 500 shots)	**83% / 77% / 77%**	10% / 0% / 0%
Updates needed to reach 90% test accuracy	**58-71**	264-298 (rarely reached)

- The barren-plateau measurement confirms the theory: with random angles, the gradient size falls about **300 times** from 1 to 16 layers; with the layer-wise starting point it stays large.
- But in actual training at this size, **Direct won** in every setting, and the difference is statistically certain (Direct better in 28-30 of 30 paired seeds, $p < 0.00001$).

Why. (1) At 10 qubits, Direct is *not yet* stuck on a plateau: its gradients are small but usable, so the problem LL fixes does not exist here. (2) Shallow circuits can only "see" a few of the input features (the light cone), so LL wastes most of its budget on circuits that are too weak. (3) LL's gradients are smaller during training, so measurement noise hurts it more.

> **Say this to sir:** "We implemented layer-wise learning and direct training for a 10-qubit, 16-layer quantum classifier on MNIST 4 vs 9 and compared them over 30 seeds in noiseless and shot-noise settings. Our gradient diagnostic confirms the barren plateau and shows that the layer-wise initialisation keeps gradients large. However, at this size direct training is not on a plateau, so it trained faster and reached higher accuracy, especially with shot noise. LL's benefit is expected only at larger scales where direct training really stalls."

2. Machine learning basics (classical)

2.1 What is classification?

A **classifier** is a function that looks at an input (here: a picture of a handwritten digit) and outputs a label (here: "4" or "9"). Because there are only two labels, this is **binary classification**. We write label 0 for digit 4 and label 1 for digit 9.

The classifier has adjustable numbers inside it, called **parameters** (or weights). **Training** means automatically choosing the parameter values so the classifier gives the right answers.

2.2 Training, validation and test data

We split the images into three groups that never overlap:

Set	Size	Used for
Training set	600 images	The model learns from these
Validation set	200 images	Choosing settings (the learning rate) before the real experiment
Test set	400 images	Final, honest score; never used for learning

The **test accuracy** (fraction of test images classified correctly) tells us how well the model works on *new* pictures it has never seen. This is what really matters.

2.3 The model outputs a probability

Our model does not output "4" or "9" directly. It outputs a probability p = "how sure am I that this is a 9", a number between 0 and 1. If p ≥ 0.5 we predict 9, otherwise 4.

To squash any number into the range 0..1 we use the **sigmoid function**: $\sigma(z) = 1 / (1 + e^{-z})$. Big positive z gives σ ≈ 1, big negative z gives σ ≈ 0, and z = 0 gives 0.5.

2.4 The loss: measuring how wrong we are

The **loss** is one number that says how wrong the model is; lower is better. We use **binary cross-entropy (BCE)**:

```
loss = -[ y * log(p) + (1 - y) * log(1 - p) ]      averaged over images
```

- If the true label is y = 1 (digit 9) and the model says p = 0.99, the loss is tiny (0.01).
- If it says p = 0.01, the loss is huge (4.6): confident and wrong is punished hard.
- A model that just guesses p = 0.5 for everything has loss ln 2 ≈ **0.693**. This is the "coin-flip" level; you will see 0.69 many times in this project.

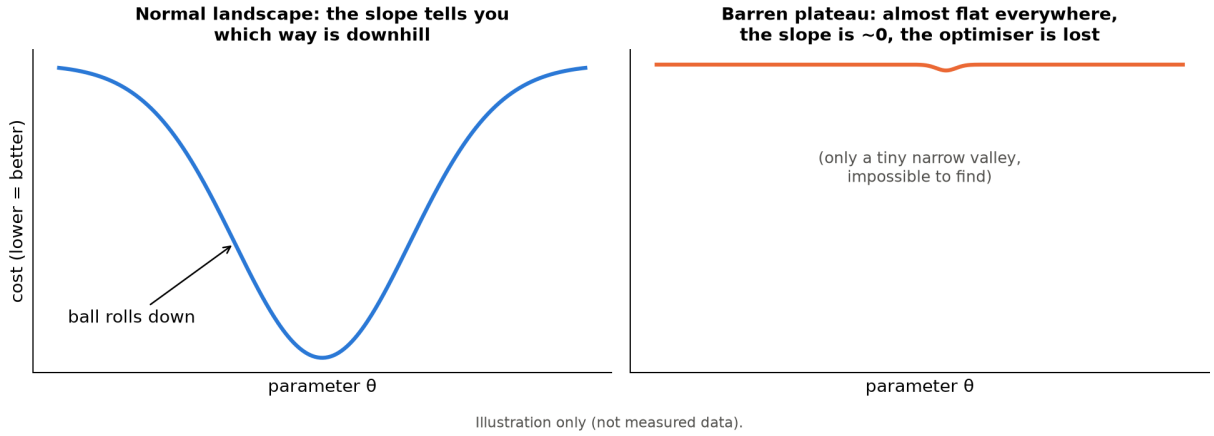
2.5 Gradient descent: walking downhill

Imagine the loss as a landscape: each position is a choice of parameters, the height is the loss. Training means walking downhill to a low point.

The **gradient** is the slope of the landscape: for each parameter θ it tells how much the loss changes if θ changes a little (written ∂loss/∂θ, "partial derivative"). Gradient descent repeats:

```
θ_new = θ_old - learning_rate * gradient
```

i.e. take a small step in the downhill direction. The **learning rate** (here 0.03) is the step size.



Left: a normal landscape where the slope guides you down. Right: a barren plateau, where the slope is nearly zero everywhere (illustration).

2.6 Mini-batches, updates and epochs

Computing the gradient on all 600 training images at every step is slow, so each step uses a random **mini-batch** of 16 images. One step = one **update** (also called an iteration). 600 / 16 = 37.5 updates go through the training set once (one **epoch**). Our budget of **320 updates** is about 8.5 epochs.

2.7 Adam: a smarter gradient descent

Adam is the most popular improved version of gradient descent. In simple terms it (1) keeps a running average of recent gradients ("momentum": keep rolling in a consistent direction, ignore jitter) and (2) scales each parameter's step by how large its gradients usually are (so every parameter moves at a sensible speed). It is the standard optimiser and the problem statement asks for "Adam or gradient descent".

2.8 Random seeds and why we use 30 of them

Training involves randomness: the starting parameter values and the order of mini-batches. A **seed** is a number that fixes this randomness, so a run can be repeated exactly. One lucky or unlucky run proves nothing, so we repeat everything with **30 different seeds** and report averages and how often training succeeds.

Say this to sir: "A classifier outputs a probability via a sigmoid; we measure error with binary cross-entropy, where 0.69 means random guessing. Training follows the gradient downhill using Adam with mini-batches of 16 images, and we repeat each experiment over 30 random seeds."

3. Quantum computing basics

3.1 Bits and qubits

A classical **bit** is either 0 or 1. A quantum bit, a **qubit**, can be in a combination of both, called a **superposition**. We write the two basic states as |0⟩ and |1⟩ (the "ket" notation |...⟩ just means "quantum state").

A useful picture: a qubit is an **arrow** (on a sphere called the Bloch sphere). Arrow pointing up = |0⟩, pointing down = |1⟩, anything in between = a superposition.

3.2 Measurement and probabilities

You cannot look at the arrow directly. When you **measure** a qubit you get 0 or 1 at random. An arrow tilted by angle θ from the top gives 0 with probability $\cos^2(\theta/2)$ and 1 with probability $\sin^2(\theta/2)$. After measuring, the superposition is gone, so to learn the probabilities you must prepare and measure the same circuit many times. Each repetition is called a **shot**.

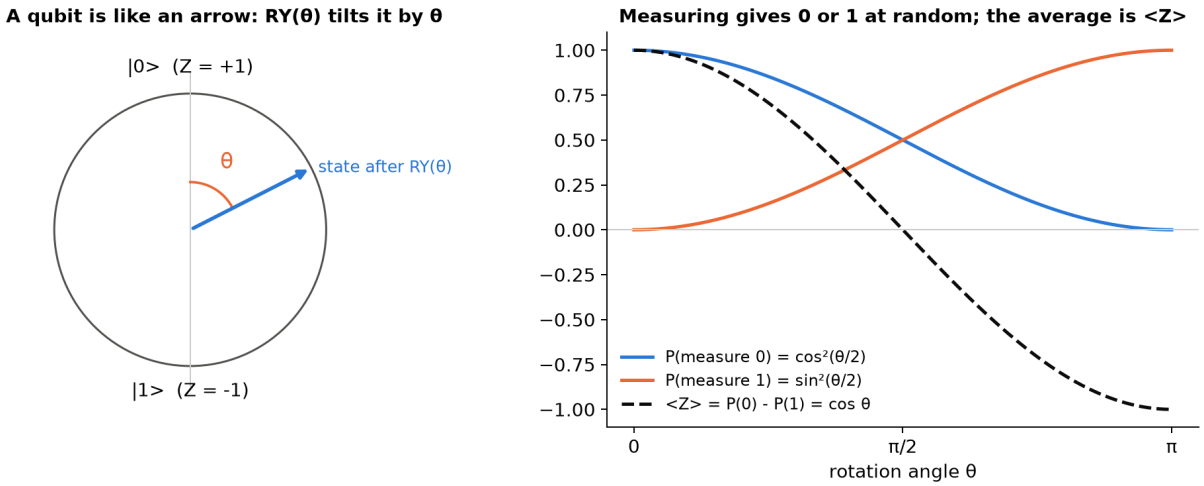
3.3 Expectation value ⟨Z⟩

We give outcome 0 the value +1 and outcome 1 the value -1. The average of these values over many shots is called the **expectation value of Z**, written ⟨Z⟩:

```
<Z> = P(0) - P(1)          always between -1 and +1
```

- Arrow up (|0⟩): ⟨Z⟩ = +1. Arrow down (|1⟩): ⟨Z⟩ = -1. Arrow sideways: ⟨Z⟩ = 0.
- For one qubit rotated by θ from the top: ⟨Z⟩ = cos θ.

With a finite number of shots S, the measured average is only an estimate. Its random error is about 1/√S: about 0.045 for 500 shots and 0.032 for 1000 shots. This is **sampling noise (shot noise)**, and it is one of the noise types in our project.



A qubit as an arrow (left). Rotating by ϑ changes the probabilities of measuring 0 or 1 and the average ⟨Z⟩ = cos ϑ (right).

3.4 Quantum gates used in this project

A **gate** is an operation that changes qubits, like a logic gate in a normal computer. We use only three:

Gate	Acts on	What it does (arrow picture)
RY(θ)	1 qubit	Rotates the arrow by angle θ around the Y axis (tilts it up/down)
RZ(θ)	1 qubit	Rotates the arrow by angle θ around the Z axis (spins it around the vertical axis); this changes the *phase*, which matters once qubits are entangled and rotated again
CZ	2 qubits	"Controlled-Z": if both qubits are 1, flip the sign of that part of the state. It creates **entanglement**

Important facts we use later:

- RY(0) and RZ(0) do nothing (they are the **identity**).
- CZ does not change the probabilities of measuring Z. It only matters when later gates rotate the qubits. In maths: CZ **commutes** with the Z measurement.

3.5 Many qubits and entanglement

With n qubits there are 2^n basic states ($|00...0\rangle$ up to $|11...1\rangle$). A general state needs 2^n numbers to describe: for our 10 qubits that is 1024 complex numbers. This exponential growth is why quantum computers might be powerful, and also why simulating them on a laptop gets impossible beyond about 30 qubits.

Entanglement means the qubits are linked: the state cannot be described qubit by qubit. CZ gates create it. Without entanglement our circuit would just be 10 independent one-qubit calculations.

3.6 How to read a circuit diagram

- Each horizontal line is one qubit (q_0 at the top), starting in $|0\rangle$ on the left.
- Time flows left to right.
- A box on a line is a one-qubit gate; two dots joined by a vertical line is a CZ.
- A meter symbol (or "measure") at the end is a measurement.

> **Say this to sir:** "A qubit is like an arrow; measuring it gives 0 or 1 at random, and the average of +1/-1 outcomes is the expectation value $\langle Z \rangle$. We use RY and RZ rotations and CZ entangling gates. With a finite number of shots, $\langle Z \rangle$ is only estimated, with error about $1/\sqrt{\text{shots}}$."

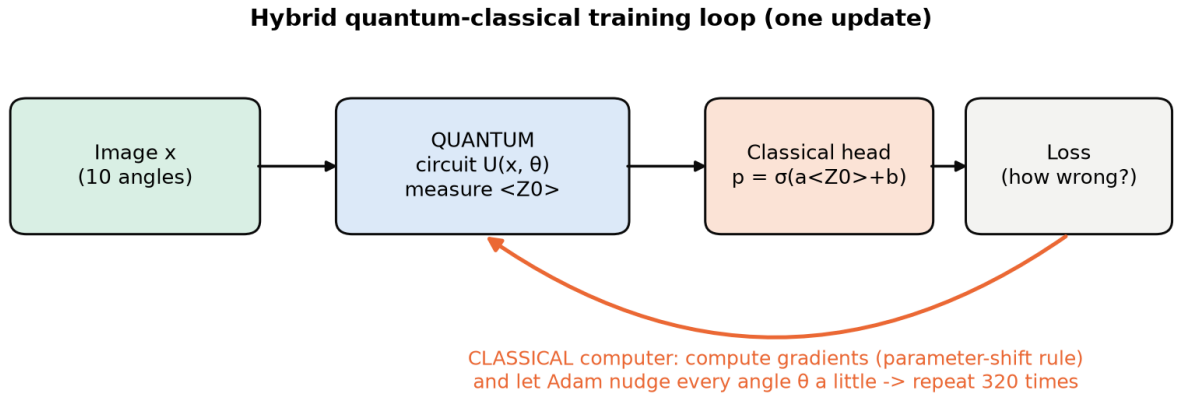
4. Variational quantum circuits (quantum neural networks)

4.1 The idea

A **variational quantum circuit (VQC)**, also called a quantum neural network (QNN), is a quantum circuit with **adjustable rotation angles** θ. It is used exactly like a classical neural network:

1. **Encode** the input data into the qubits (here: rotate qubit q by the q-th feature of the image).
2. Apply **trainable layers** of gates whose angles θ are the parameters.
3. **Measure** to get a number (here $\langle Z_0 \rangle$, the expectation of Z on qubit 0).
4. Turn that number into a prediction, compute the loss.
5. A **classical computer** computes gradients and updates θ (with Adam). Repeat.

Because a quantum part and a classical part work together in a loop, this is called a **hybrid quantum-classical** algorithm. It is the main kind of algorithm for today's small, noisy quantum computers (called NISQ devices).



One training update of the hybrid loop. The quantum computer evaluates the circuit; the classical computer computes the loss and updates the angles.

4.2 Layers and depth

A **layer** is one repeating block of gates: in our case, an RY and an RZ rotation on every qubit followed by a ring of CZ gates. The number of layers is the **depth**. A deeper circuit has more parameters and can represent more complicated functions, just like a deeper neural network.

4.3 How gradients are measured on a quantum computer: the parameter-shift rule

On a real quantum computer we cannot "look inside" to differentiate. Instead we use the **parameter-shift rule**. For our rotation gates it says, *exactly*:

```
d f / d theta  =  [ f(theta + pi/2)  -  f(theta - pi/2) ] / 2
```

where f is the measured expectation value. So to get the gradient for one angle we run the circuit two more times, once with that angle shifted up by 90° and once shifted down by 90°, and take half the difference.

Worked example with one qubit: $f(\theta) = \langle Z \rangle = \cos \theta$. At $\theta = 0$: $[\cos(90°) - \cos(-90°)] / 2 = (0 - 0)/2 = 0$, and the true derivative $-\sin(0) = 0$. At $\theta = 90°$: $[\cos 180° - \cos 0°]/2 = (-1 - 1)/2 = -1$, and $-\sin 90° = -1$. Exact.

Cost: with P trainable angles and a batch of 16 images, one gradient needs $16 \times (1 + 2P)$ circuit runs. For Direct ($P = 320$) that is **10,256 circuit runs per update**, each repeated 500 or 1000 times (shots). This is why shot-noise training is so slow to simulate.

> **Say this to sir:** "A VQC is a parameterised quantum circuit trained in a hybrid loop: the quantum device evaluates $\langle Z_0 \rangle$, the classical computer computes the loss and updates the angles with Adam. Gradients on hardware come from the parameter-shift rule: run the circuit with the angle shifted by $\pm\pi/2$ and take half the difference."

5. Barren plateaus

5.1 What a barren plateau is

A **barren plateau** is a training failure: the loss landscape becomes almost completely flat. The gradient for every parameter is almost exactly zero, so the optimiser cannot tell which direction is downhill.

Researchers (McClean et al., 2018) proved that for circuits that are deep enough and start from **random** angles, the gradients are typically zero on average and their **variance** (their typical size squared) shrinks **exponentially** with the number of qubits: roughly like $1/2^n$. With 50 qubits that is about 10^{-15}: unmeasurable.

5.2 Why it happens (intuition)

A deep circuit with random angles scrambles the state so thoroughly that it behaves like a completely random quantum state. In a random state of n qubits, every measurement average is extremely close to its mean value (a "concentration" effect, like how the average of many coin flips is always close to 0.5). If the output hardly depends on the angles, the slope with respect to each angle is tiny.

5.3 Why it is so bad with real hardware

On real hardware, gradients come from measurements with shot noise of about $1/\sqrt{S}$. If the true gradient is 0.0001, you would need around 100 million shots to even see it above the noise. So in practice the optimiser just walks randomly. **The signal-to-noise ratio** (gradient size divided by noise size) is what matters.

5.4 What makes it better or worse

Factor	Effect
More qubits	Worse (exponentially)
Deeper circuit with random angles	Worse (until it saturates)
Global cost (measure Z on all qubits and multiply)	Very bad, even for shallow circuits
Local cost (measure Z on one qubit, like our $\langle Z_0 \rangle$)	Much better for shallow circuits
Starting near the identity / shallow	Better: gradients start large

5.5 How we measure it in this project

We pick 100 random sets of angles, compute the exact gradient for each, and compute the **variance** of the gradient across these 100 random starts. A large variance means gradients are typically large (good); a tiny variance means a barren plateau. We do this for different numbers of qubits and different depths (Chapter 24, Figure 3).

> **Say this to sir:** "A barren plateau means the gradient variance of a randomly initialised deep circuit shrinks exponentially with the number of qubits, so gradients are effectively zero and drown in shot noise. It is worst for global costs, many qubits and deep random circuits."

6. Layer-wise learning (the proposed fix)

6.1 The idea (Skolik et al., 2021)

Instead of training the deep circuit from a random start, **grow** it:

1. Start with a **shallow** circuit (1 layer). Shallow circuits have large gradients, so training can start immediately.
2. Train it for a while.
3. **Add a new layer** at the end whose angles are (almost) zero, so the new layer is (almost) the identity and the circuit's output does not change. Nothing learned is lost.
4. Continue training with the extra layer. Repeat until the full depth is reached.

6.2 Why it should help

- At every moment the circuit is "effectively shallow": the new layers start near identity, so the state is never scrambled randomly, and the gradients stay large.
- Larger gradients give a better **signal-to-noise ratio** under shot noise, so each noisy update is more useful.
- In the paper, LL reached lower test error and more successful runs than direct training for their circuits, especially with few shots.

6.3 Two variants

- **Frozen LL** (the original paper): when a new layer is added, only the newest layers train; older layers are frozen, and in a second phase larger groups are trained.
- **Joint LL** (what we use): when a new layer is added, *all* layers present so far keep training. This matches the brief ("start with only the first layer trained, then successively add layers after partial training") and is simpler to explain. (An earlier version of the project also tested the frozen variant; it was not better.)

> **Say this to sir:** "Layer-wise learning starts with one layer and adds near-identity layers during training. The circuit is always effectively shallow, so gradients stay large and the signal-to-noise ratio is better. We use the joint variant where all present layers keep training."

7. The problem statement and our objectives

The project brief from sir asks us to design, implement and evaluate a layer-wise learning strategy for variational circuits to mitigate barren plateaus, and compare it with direct training of a fixed full-depth circuit.

Brief asks	What we did	Where in this guide
O1: choose a QML task, e.g. small digit classification	Real MNIST digits 4 vs 9, 10 PCA features, 10 qubits, 16 layers	Ch. 8-9
O2: (a) Direct training of a fixed-depth ansatz	All 16 layers trained from step 0, random angles	Ch. 10
O2: (b) Layerwise training	Start with 1 layer, add a near-identity layer every 20 updates	Ch. 10
O3: compare final accuracy	Test accuracy, 30 seeds, with 95% confidence intervals	Ch. 23-24
O3: number of iterations to converge	Updates until test accuracy first reaches 90%	Ch. 12, 24
O3: sensitivity to sampling noise (limited shots)	Full training with 1000 and 500 shots	Ch. 11, 24
Code: interactive notebook, standard optimiser	`notebook/layerwise_vs_direct.ipynb`, Adam	Ch. 22, 30
Results: plots LL vs Direct, test accuracy vs iterations, % successful runs	Figures 4-7	Ch. 24
Analysis: explain gradients / signal-to-noise	Barren-plateau diagnostic (Figure 3) and analysis	Ch. 24-27

The brief also states an *expected* outcome (LL better for deep circuits). Our experiment tested this honestly; the data showed the opposite at our scale, and Chapter 26 explains why. Science reports what the data shows.

> **Say this to sir:** "All three objectives are covered: the MNIST 4-vs-9 task, both training protocols with the same budget, and a comparison of accuracy, convergence speed and shot-noise sensitivity over 30 seeds, plus a barren-plateau diagnostic for the analysis."

8. The task and the data (Objective O1)

8.1 MNIST, digits 4 vs 9

MNIST is the most famous dataset in machine learning: 70,000 small (28 × 28 pixel) grey-scale pictures of handwritten digits 0-9, with the correct label for each. 60,000 are the official training images and 10,000 the official test images. We use the real dataset (downloaded from OpenML, stored in `data/mnist_784.npz`).

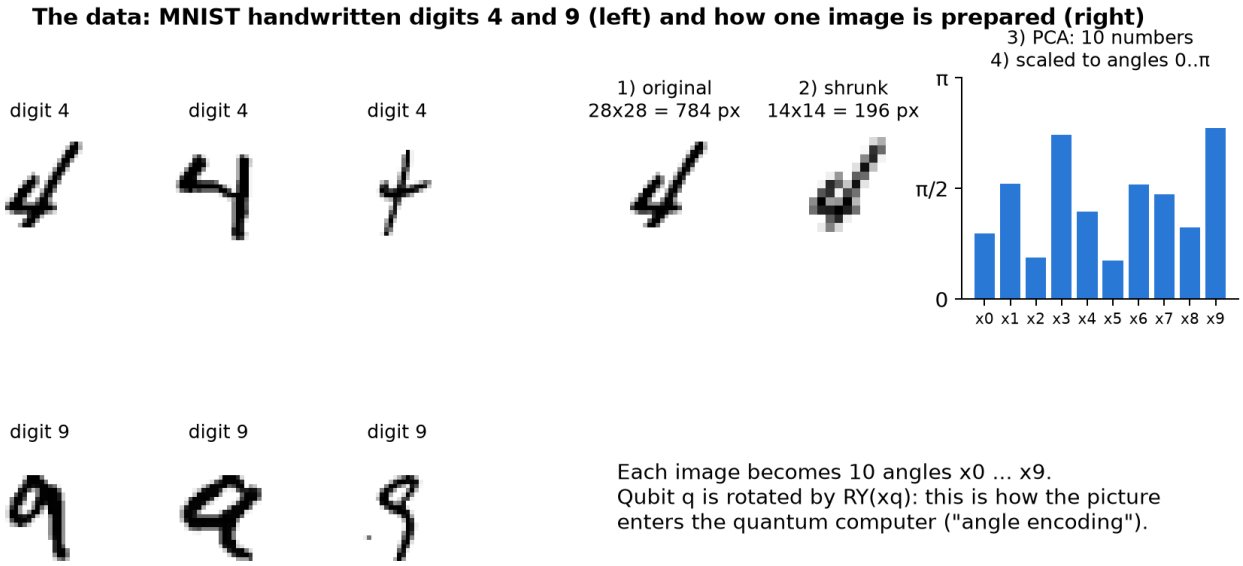
We classify **4 vs 9**. These two digits look alike (a closed or open top with a vertical stroke), so this is one of the hardest digit pairs. An easy pair like 0 vs 1 would be solved by almost any model and would not show differences between training methods.

8.2 From a picture to 10 angles

A quantum computer with 10 qubits cannot take 784 pixels directly. We compress each image in four steps:

1. **Shrink** 28 × 28 to 14 × 14 by averaging each 2 × 2 block of pixels (784 → 196 numbers). Pixel values are divided by 255 so they lie in 0..1.
2. **PCA** (Principal Component Analysis) to **10 numbers**. PCA finds the 10 directions in which the images differ the most, and describes every image by 10 numbers along those directions. These 10 numbers keep **64.6%** of all the variation in the images.
3. **Scale** each of the 10 numbers to an angle between 0 and π (MinMax scaling), because they will be used as rotation angles.
4. **Encode**: qubit q is rotated by RY(x_q). This is called **angle encoding**. One feature per qubit, so 10 features need 10 qubits.

PCA and the scaling are learned from the **training images only** and then applied to validation and test images. Using test images there would be "cheating" (data leakage).



Real examples of 4s and 9s (left) and the four preparation steps for one image (right).

8.3 The three subsets

Subset	Images	Taken from	Balance
Train	600	official MNIST training part	300 fours + 300 nines
Validation	200	official MNIST training part (different images)	100 + 100
Test	400	official MNIST test part	200 + 200

The subsets are **balanced** (equal numbers of each digit), so 50% accuracy means guessing. The images are drawn with a fixed data seed (1234), so **every run of every method uses exactly the same images**. The code checks that no image appears in two subsets.

> **Say this to sir:** "We use real MNIST digits 4 versus 9, a hard pair. Each image is shrunk to 14×14, compressed by PCA to 10 features that keep 65% of the variance, scaled to angles 0 to π, and encoded with one RY rotation per qubit. We use 600 training, 200 validation and 400 test images, balanced and disjoint, identical for every run."

9. The quantum classifier (our model)

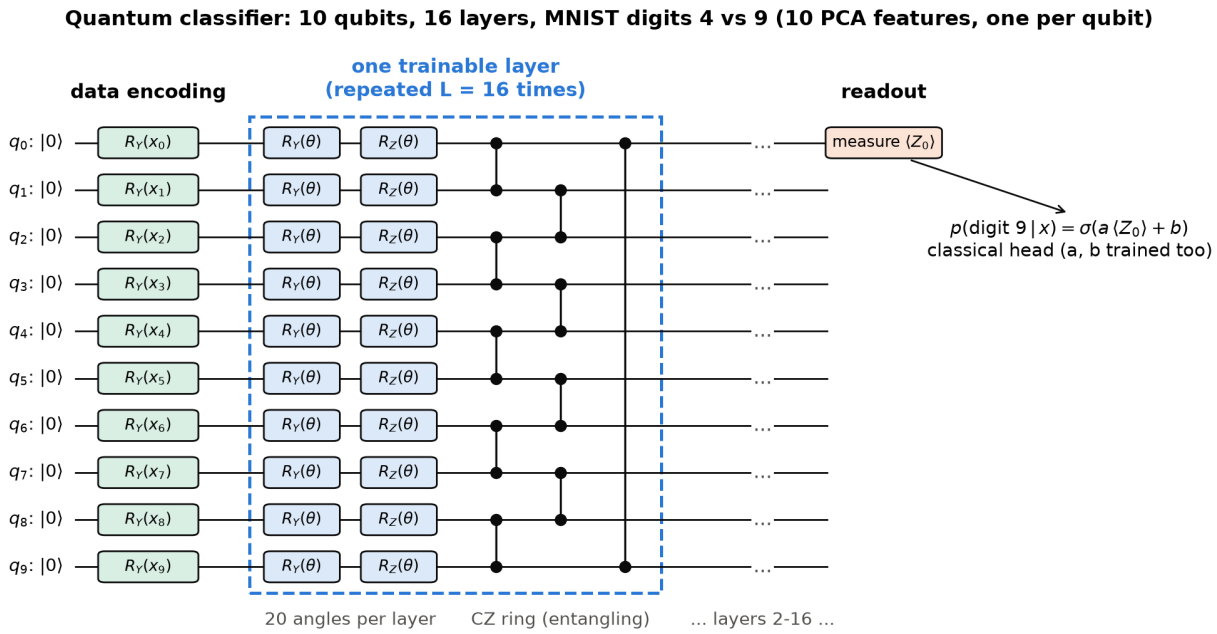


Figure 1. The quantum classifier. Green: data encoding; blue dashed box: one trainable layer, repeated 16 times; orange: readout and classical head.

9.1 The circuit, part by part

Part	Gates	Meaning	
Start	all 10 qubits in	$\|0\rangle$	arrows all pointing up
Encoding	RY(x_q) on qubit q	the image enters the circuit: feature q tilts qubit q	
Trainable layer (16 times)	RY(θ), RZ(θ) on every qubit, then CZ on pairs (0,1), (1,2), ..., (8,9), (9,0)	20 adjustable angles per layer, then entangle neighbours in a ring	
Readout	measure Z on qubit 0	gives $\langle Z_0 \rangle$, a number between -1 and +1	
Classical head	$p = \sigma(a \cdot \langle Z_0 \rangle + b)$	turns $\langle Z_0 \rangle$ into a probability of "digit 9"; a and b are two ordinary trainable numbers	

Number of parameters: 16 layers × 10 qubits × 2 angles = **320 quantum angles**, plus a and b = 322.

9.2 Why these design choices

- **RY + RZ per qubit:** RY alone keeps all numbers real; adding RZ lets the circuit use complex phases, which makes the circuit more expressive. This RY-RZ + entangler pattern is a standard "hardware-efficient ansatz" (ansatz = the chosen circuit shape).
- **CZ ring:** connects every qubit to its two neighbours, like a chain closed into a circle. It is cheap on real hardware (only nearest neighbours).

- **Local readout $\langle Z_0 \rangle$:** measuring only qubit 0 is a *local cost*. Theory says local costs are trainable at moderate depth, while global costs (measuring all qubits) have barren plateaus even when shallow. Using a global cost would make *both* methods fail, which would not be an interesting comparison.
- **Why CZ is special for layer-wise learning:** CZ commutes with Z. A new layer with all angles 0 is RY(0)=I, RZ(0)=I and then a CZ ring, which cannot change $\langle Z_0 \rangle$. So a zero-angle layer added at the end leaves the output *exactly* unchanged (we verified: change = 0.0). This is what "start new layers near identity" needs.
- **Classical head a, b:** $\langle Z_0 \rangle$ is between -1 and 1; scaling by a and shifting by b lets the sigmoid produce confident probabilities and corrects any offset.

9.3 The light cone (important for the analysis)

Measuring only qubit 0 has a consequence. Working backwards from the measurement: the last layer's CZ gates do not affect $\langle Z_0 \rangle$ (they commute with Z), so a 1-layer circuit's output depends only on qubit 0, i.e. only on feature x_0. Each additional layer lets information travel one more step around the ring (to qubits 1 and 9, then 2 and 8, ...). A depth-d circuit's output therefore depends on only **2d - 1** of the 10 features:

Depth d	Features that can influence the output
1	1 (x_0 only)
2	3
3	5
5	9
6 or more	all 10

We verified this numerically. This "light cone" is a key reason why shallow circuits are weak classifiers here (Chapter 26).

> **Say this to sir:** "Our model angle-encodes 10 features, applies 16 layers of RY and RZ rotations with a CZ ring, measures $\langle Z_0 \rangle$ and feeds it to a sigmoid head, giving 320 quantum angles plus 2 classical parameters. We use a local readout because global costs have barren plateaus even when shallow, and CZ entanglers because a zero-angle layer then leaves the output exactly unchanged."

10. The two training protocols (Objective O2)

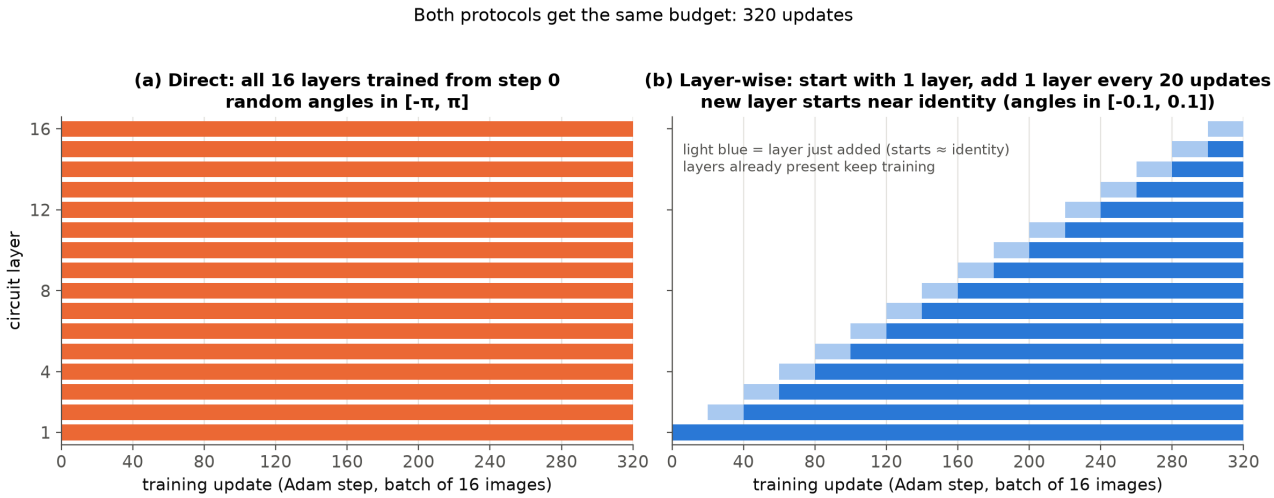


Figure 2. Which layers are trained at which update. Each bar is one layer. Left: Direct. Right: Layer-wise; light blue marks the first 20 updates of a newly added layer.

10.1 Direct training

- All 16 layers exist and are trained from update 1 to update 320.
- Every angle starts **random**, uniformly between -π and π. This is the standard initialisation and the one that theory says leads to barren plateaus.

10.2 Layer-wise training (LL)

- **Stage 1** (updates 1-20): the circuit has only layer 1. Its angles start with *exactly the same* random values as Direct's layer 1 for the same seed.
- **Stage 2** (updates 21-40): layer 2 is appended. Its angles start small, uniformly between -0.1 and 0.1, so the layer is almost the identity. Layers 1 and 2 are trained.
- ... and so on: every 20 updates one layer is added, and all layers present keep training.
- **Stage 16** (updates 301-320): the full 16-layer circuit is trained.
- 16 stages × 20 updates = **320 updates**, the same as Direct.

Why small random angles (±0.1) instead of exactly 0? The layer stays almost the identity (output almost unchanged) while each new gate gets a slightly different starting point. The original paper used zero; the difference is minor.

10.3 What makes the comparison fair

Same for both methods	
Data	the same 600/200/400 images
Circuit at full depth	identical (16 layers)
Budget	320 Adam updates, batch size 16
Learning rate	0.03 for both (chosen beforehand on validation data from 0.01 / 0.03 / 0.1)

Same for both methods	
Seed k	same first-layer angles and the same sequence of mini-batches (a *paired* comparison)

One difference to know about: with shots, LL is *cheaper*. A shallow circuit has fewer angles, so the parameter-shift rule needs fewer circuit runs. Over a whole run LL used about 1.75 million circuit executions versus 3.28 million for Direct. So we matched the number of updates (the brief's "iterations"), not the number of circuit executions.

Say this to sir: "Direct trains all 16 layers from random angles for 320 updates. Layer-wise starts with one layer, with the same random first layer as Direct, and every 20 updates appends a near-identity layer with angles in ±0.1, keeping all layers trainable, until 16 layers after 320 updates. Same data, learning rate, budget and paired seeds."

11. Noise: training with a limited number of shots

On a real quantum computer, $\langle Z_o \rangle$ is never known exactly; it is estimated from S shots. The brief asks for sensitivity to **sampling noise**. We trained both methods in three settings:

Setting	How gradients are computed	Noise in each circuit value
Noiseless	exact (automatic differentiation of the simulated circuit)	none
1000 shots	parameter-shift rule; *every* circuit value (normal and shifted) estimated from 1000 shots	about ±0.03
500 shots	same with 500 shots	about ±0.045

How a shot estimate is simulated: if the exact value is f, the probability of measuring +1 is (1 + f)/2. The number of +1 outcomes in S shots is drawn from a **binomial distribution**, and the estimate is 2k/S - 1. This is mathematically exactly what a real device produces (for sampling noise; hardware errors like decoherence are not included).

The loss and the gradients during training are computed from these noisy values, so the optimiser sees exactly what it would see on hardware. **Accuracy for the results is computed exactly**: noise affects how well the model is *trained*, and we then measure the true quality of the trained model.

> **Say this to sir:** "For the noise study we trained with the parameter-shift rule where every circuit value is estimated from 1000 or 500 shots, simulated with binomial sampling, which is exactly the statistics of a real device. Accuracy is then evaluated exactly, so we measure how much the noise damaged training."

12. How we measure performance

Metric	Definition	Why
Final test accuracy	Fraction of the 400 test images classified correctly after update 320	The main quality measure (brief: "final accuracy")
95% confidence interval (CI)	mean ± 1.96 × standard deviation / √30	How precisely we know the mean over 30 seeds
Successful run	final training loss < 0.3466 **and** final test accuracy ≥ 90%	Brief: "percentage of successful runs"
Iterations to converge	the first update at which test accuracy reaches 90%	Brief: "number of iterations to converge"
Success rate	successful runs / 30	

Where 0.3466 comes from: it is ln(2)/2, half the loss of a coin-flip classifier (0.693). Requiring both a low training loss *and* high test accuracy means a run counts as successful only if it really learned and generalises.

Why 90%: classical methods on the same 10 features reach 91.7% (logistic regression) and 95.8% (a support vector machine) test accuracy. So 90% means "the quantum model learned the task about as well as a simple classical model".

Significance tests (to check the difference is not luck):

- **Wilcoxon signed-rank test** on paired accuracies (seed k of Direct vs seed k of LL). It asks: "is one method consistently better?" without assuming a bell curve.
- **Fisher's exact test** on success counts (e.g. 25/30 vs 3/30).
- A **p-value** below 0.05 means the difference is very unlikely to be chance; ours are below 0.00001.

> **Say this to sir:** "We report final test accuracy with 95% confidence intervals, the success rate where success means training loss below ln2/2 and test accuracy at least 90%, and iterations to converge, defined as the first update reaching 90% test accuracy. Significance is checked with a paired Wilcoxon test and Fisher's exact test."

13. The experiment as a whole

Methods	2 (Direct, Layer-wise)
Noise settings	3 (noiseless, 1000 shots, 500 shots)
Seeds	30 per method and setting (seeds 0-29)
Total training runs	2 × 3 × 30 = **180**
Updates per run	320
Average time per run (8-core laptop)	noiseless: Direct 210 s, LL 119 s; with shots: Direct ~765 s, LL ~290 s
Total compute	about 5 hours, runs in parallel on all CPU cores
Extra experiment	barren-plateau gradient diagnostic (100 random initialisations per point)

Every run is saved as its own file, `results/runs/<setting>/<method>/seed_XXX.json`, containing the settings, the full training history (accuracy, loss, gradient size after every update), a summary and the final trained parameters. All graphs and numbers are computed from these files.

Why a fast simulator? The model is defined in PennyLane (the standard quantum ML library). Running 180 × 320 updates through PennyLane's general simulator, with shots and parameter shifts, would take weeks. So we wrote a specialised simulator of *exactly the same circuit* in PyTorch (`src/fastsim.py`) and checked that it agrees with PennyLane to 15 decimal places.

> **Say this to sir:** "In total 180 runs: 2 methods, 3 noise settings, 30 seeds, each 320 updates, about 5 hours on a laptop. Each run is saved with its full history and final parameters, and all figures are computed from these files. Training uses a fast PyTorch simulator verified against PennyLane."

14. The code: overview

14.1 The folder

```
quantum comp 2.0/
  README.md                 short written explanation
  run_all.py                one command per step (train / diagnostic / figures)
  requirements.txt          list of Python packages
  notebook/
    layerwise_vs_direct.ipynb   the interactive notebook (deliverable)
  src/                      all the program code
    config.py               every setting in one place
    data.py                 loading and preparing MNIST
    circuit.py              the quantum circuit in PennyLane (reference model)
    fastsim.py              fast simulator of the same circuit + shot sampling
    train.py                Direct and Layer-wise training, Adam, shot gradients
    runner.py               runs many training jobs in parallel and saves them
    gradients.py            barren-plateau diagnostic
    figures.py              results table and all 7 graphs
  results/
    runs/noiseless|shots1000|shots500/direct|ll/seed_000..029.json   (180 runs)
    diagnostic/bp_diagnostic.json      barren-plateau measurements
    summary.csv                        the results table
  figures/                  fig1 ... fig7 (PNG)
  data/mnist_784.npz        the real MNIST data
```

14.2 How the files work together

1. `run_all.py train` asks `runner.py` to create 180 jobs.
2. Each job calls `train.train_run(...)`, which gets the data from `data.py`, builds the model with the simulator from `fastsim.py`, trains it with Adam and returns the full record.
3. `runner.py` saves each record as a JSON file in `results/runs/`.
4. `run_all.py diagnostic` runs `gradients.py` and saves `bp_diagnostic.json`.
5. `run_all.py figures` runs `figures.py`, which reads all saved files and draws the graphs and the table.
6. The notebook imports the same modules, checks the simulator against `circuit.py` (PennyLane), and shows everything.

14.3 Python in 1 minute (enough to read the code)

You see	It means
`x = 5`	store 5 in a variable named x
`def f(a, b):`	define a function f with inputs a, b; the indented lines below are its body
`return y`	the function's output
`class Model:`	a "class" bundles data and functions (methods) together; `self` is the object itself
`for i in range(16):`	repeat the indented block for i = 0, 1, ..., 15
`if ...: ... else: ...`	do one thing or another
`[a, b, c]`	a list; `w[0]` is the first item; `w[:3]` the first three
`dict(a=1)` or `{"a": 1}`	a dictionary: values looked up by name, `cfg["lr"]`
`np.` / `torch.`	numpy and PyTorch: libraries for fast maths on arrays

You see	It means
`"""text"""` or `# text`	comments/explanations, ignored by the computer
`import x` / `from x import y`	use code from another file or library

Array shapes: the angles are stored in an array `w` of shape (16, 10, 2): `w[l, q, 0]` is the RY angle of layer l on qubit q and `w[l, q, 1]` is the RZ angle. Python counts from 0, so "layer 1" in the text is `w[0]` in code.

In the listings below, the numbers on the left are line numbers in the real file.

15. config.py: all settings

This file holds every number that defines the experiment, so nothing is hidden inside other files.

src/config.py (lines 1-27)

```
"""All experiment settings (fixed before the comparison runs)."""
from __future__ import annotations

from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
RESULTS = ROOT / "results"
FIGURES = ROOT / "figures"

# The task: real MNIST digits 4 vs 9, 10 qubits, 16 layers, readout <Z_0> (local cost)
TASK = dict(n_qubits=10, n_layers=16, digits=(4, 9), cost="local")
DATA = dict(n_train=600, n_val=200, n_test=400)

TRAIN = dict(
    batch_size=16,          # images per update
    updates_per_stage=20,   # LL adds a layer every 20 updates -> 16 x 20 = 320 updates
    lr=0.03,                # Adam learning rate (chosen on validation data in a pilot sweep: 0.01 / 0.03 / 0.1)
    init_range=3.141592653589793,   # Direct: every angle ~ U(-pi, pi)
    ident_range=0.1,        # LL: added layers start near identity, angles ~ U(-0.1, 0.1)
)

METHODS = ("direct", "ll")  # ll = layer-wise learning (all present layers keep training)
SEEDS = tuple(range(30))
SHOTS_TRAIN = (500, 1000)   # finite-shot (sampling-noise) training

LOSS_THRESHOLD = 0.3466     # ln(2)/2: half the loss of a coin-flip classifier
ACC_TARGET = 0.90           # success = final train loss < threshold AND test accuracy >= 0.90
```

Explained:

- Lines 6-8: folder locations (the project folder, results/, figures/).
- Line 11: the task: 10 qubits, 16 layers, digits 4 and 9, local cost (measure only qubit 0).
- Line 12: 600 training, 200 validation, 400 test images.
- Lines 14-20: training settings: batch size 16; LL adds a layer every 20 updates (so 16 × 20 = 320 updates total); Adam learning rate 0.03; Direct starts angles in ±π; LL's new layers start in ±0.1.
- Line 22: the two methods, "direct" and "ll".
- Line 23: seeds 0-29.
- Line 24: the shot numbers used for noisy training.
- Lines 26-27: the success rule: final training loss below 0.3466 (= ln2/2) and test accuracy at least 90%.

16. data.py: preparing MNIST

16.1 Loading the data

src/data.py (lines 1-55)

```
"""Real MNIST loading and preprocessing.

Source: OpenML dataset "mnist_784" version 1 (id 554), fetched with
sklearn.datasets.fetch_openml and cached under data/. Rows 0..59999 are the
official MNIST training images and rows 60000..69999 the official test images.
Train/validation images are drawn from the official training part and test
images from the official test part, so the three subsets are disjoint by
construction (and this is additionally asserted on image indices).

No synthetic data is ever generated here.
"""
from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass
from pathlib import Path

import numpy as np
from sklearn.decomposition import PCA
from sklearn.preprocessing import MinMaxScaler

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
CACHE = DATA_DIR / "mnist_784.npz"
N_OFFICIAL_TRAIN = 60000

# Fixed seed for drawing the data subset. It is the same for every training
# seed and every method, so methods are always compared on identical data.
DATA_SEED = 1234


def load_mnist() -> tuple[np.ndarray, np.ndarray, dict]:
    """Return (X uint8 [70000, 784], y int64 [70000], provenance dict)."""
    if not CACHE.exists():
        from sklearn.datasets import fetch_openml

        X, y = fetch_openml("mnist_784", version=1, data_home=str(DATA_DIR),
                            as_frame=False, return_X_y=True, parser="liac-arff")
        np.savez_compressed(CACHE, X=X.astype(np.uint8), y=y.astype(np.int64))
    with np.load(CACHE) as f:
        X, y = f["X"], f["y"]
    assert X.shape == (70000, 784) and y.shape == (70000,), "unexpected MNIST shape"
    assert X.max() == 255 and X.min() == 0
    sha = hashlib.sha256(CACHE.read_bytes()).hexdigest()
    prov = {
        "source": "OpenML mnist_784 v1 (id 554) via sklearn.datasets.fetch_openml",
        "cache_file": str(CACHE.relative_to(ROOT)),
        "cache_sha256": sha,
        "shape": list(X.shape),
        "official_train_rows": "0..59999",
        "official_test_rows": "60000..69999",
    }
    return X, y, prov

```

- Lines 1-11: the description. Important: only real MNIST is used; train/validation come from the official training part and test from the official test part.
- Line 30: `DATA_SEED = 1234` fixes which images are chosen, so all runs use the same images.
- Lines 33-55 `load_mnist`: if the file `data/mnist_784.npz` does not exist, download MNIST from OpenML (line 36-40) and save it. Then load the 70,000 images (`X`, each 784 pixel values 0-255) and

labels (y). It checks the shape, and records a fingerprint (SHA-256 hash) of the file so we can prove which data was used.

16.2 Shrinking images and choosing balanced subsets

src/data.py (lines 57-83)

```
57  def downsample(images: np.ndarray, factor: int = 2) -> np.ndarray:
58      """Resize 28x28 -> (28/factor)^2 by block averaging; returns floats in [0,1]."""
59      n = images.shape[0]
60      side = 28 // factor
61      im = images.reshape(n, side, factor, side, factor).astype(np.float64) / 255.0
62      return im.mean(axis=(2, 4)).reshape(n, side * side)
63
64
65  @dataclass
66  class Split:
67      x_train: np.ndarray
68      y_train: np.ndarray
69      x_val: np.ndarray
70      y_val: np.ndarray
71      x_test: np.ndarray
72      y_test: np.ndarray
73      info: dict
74
75
76  def _balanced_indices(y, rows, digits, per_class, rng, exclude=()):
77      out = []
78      excl = set(int(i) for i in exclude)
79      for d in digits:
80          cand = np.array([i for i in rows[y[rows] == d] if int(i) not in excl])
81          out.append(rng.choice(cand, size=per_class, replace=False))
82      return np.concatenate(out)
83
```

- `downsample` (lines 57-63): reshapes each 28 × 28 image into 14 × 14 blocks of 2 × 2 pixels and averages each block; dividing by 255 puts values in 0..1.
- `Split` (lines 65-73): a simple container holding the six arrays (train/val/test images and labels) and an `info` dictionary.
- `_balanced_indices` (lines 76-82): for each of the two digits, randomly pick `per_class` image numbers, optionally excluding ones already used. This guarantees equal numbers of 4s and 9s.

16.3 The full preparation pipeline

src/data.py (lines 85-130)

```
85  def make_split(digits=(0, 1), n_features=8, n_train=600, n_val=200, n_test=400,
86                 angle_range=(0.0, np.pi), data_seed=DATA_SEED) -> Split:
87      """Balanced binary MNIST subsets -> downsample -> PCA -> MinMax to angles.
88
89      PCA and the MinMax scaler are fitted on the training subset only.
90      Label 1 = digits[1], label 0 = digits[0].
91      """
92      X, y, prov = load_mnist()
93      rng = np.random.default_rng(data_seed)
94      tr_rows = np.arange(N_OFFICIAL_TRAIN)
95      te_rows = np.arange(N_OFFICIAL_TRAIN, len(y))
96      idx_tr = _balanced_indices(y, tr_rows, digits, n_train // 2, rng)
97      idx_va = _balanced_indices(y, tr_rows, digits, n_val // 2, rng, exclude=idx_tr)
98      idx_te = _balanced_indices(y, te_rows, digits, n_test // 2, rng)
99      # disjointness checks (indices and official partition)
100     assert len(set(idx_tr) & set(idx_va)) == 0
101     assert len(set(idx_tr) & set(idx_te)) == 0 and len(set(idx_va) & set(idx_te)) == 0
102     assert idx_te.min() >= N_OFFICIAL_TRAIN and max(idx_tr.max(), idx_va.max()) < N_OFFICIAL_TRAIN
103     # shuffle order within each subset (deterministic)
104     idx_tr, idx_va, idx_te = (rng.permutation(a) for a in (idx_tr, idx_va, idx_te))
```

```
105
106     small = {k: downsample(X[i]) for k, i in (("tr", idx_tr), ("va", idx_va), ("te", idx_te))}
107     pca = PCA(n_components=n_features, random_state=0).fit(small["tr"])
108     feats = {k: pca.transform(v) for k, v in small.items()}
109     scaler = MinMaxScaler(feature_range=angle_range).fit(feats["tr"])
110     ang = {k: np.clip(scaler.transform(v), *angle_range) for k, v in feats.items()}
111     lab = {k: (y[i] == digits[1]).astype(np.float64)
112            for k, i in (("tr", idx_tr), ("va", idx_va), ("te", idx_te))}
113
114     info = {
115         "provenance": prov,
116         "digits": list(digits),
117         "label_map": {str(digits[0]): 0, str(digits[1]): 1},
118         "n_features": n_features,
119         "preprocessing": "28x28 -> 14x14 block-average, /255, PCA(fit train) -> "
120                          f"{n_features} comps, MinMax(fit train) to [{angle_range[0]:.4f}, "
121                          f"{angle_range[1]:.4f}] rad, clipped",
122         "pca_explained_variance": float(pca.explained_variance_ratio_.sum()),
123         "sizes": {"train": len(idx_tr), "val": len(idx_va), "test": len(idx_te)},
124         "class_counts": {k: {str(d): int((y[i] == d).sum()) for d in digits}
125                          for k, i in (("train", idx_tr), ("val", idx_va), ("test", idx_te))},
126         "data_seed": data_seed,
127         "index_sha256": hashlib.sha256(
128             json.dumps([idx_tr.tolist(), idx_va.tolist(), idx_te.tolist()]).encode()).hexdigest(),
129     }
130     return Split(ang["tr"], lab["tr"], ang["va"], lab["va"], ang["te"], lab["te"], info)
```

- Lines 92-98: choose 600 train and 200 validation images from the official training rows (validation excludes train), and 400 test images from the official test rows.
- Lines 100-102: **safety checks**: no image is in two subsets, and test images really come from the official test part. If any check fails, the program stops.
- Line 104: shuffle the order within each subset.
- Line 106: shrink all images to 14 × 14.
- Line 107: **PCA is fitted on the training images only**, keeping 10 components.
- Line 108: every subset is transformed with that same PCA.
- Lines 109-110: MinMax scaler (also fitted on training only) maps each feature to the range [0, π]; test values slightly outside are clipped.
- Lines 111-112: labels: 1 if the digit is 9, else 0.
- Lines 114-129: a record of everything (source, sizes, class counts, explained variance 64.6%, a hash of the chosen image indices). Every saved run contains this, which is how we verified all 180 runs used the identical split.

The last function in the file, `raw_images` (lines 133-142), returns the original pictures of the same training images; it is only used to draw example digits.

> **Say this to sir:** "data.py downloads real MNIST, picks balanced, disjoint subsets with a fixed seed, shrinks images to 14×14, fits PCA and scaling on the training set only, and maps 10 features to angles in [0, π]. It asserts the subsets don't overlap."

17. circuit.py: the model in PennyLane

PennyLane is the standard Python library for quantum machine learning (by Xanadu). A **QNode** is PennyLane's object for "a quantum circuit that can be run and differentiated like a normal function".

src/circuit.py (lines 1-63)

```
"""The variational quantum circuit (PennyLane).

Architecture (per sample x with n features, one per qubit):
    AngleEmbedding(x, rotation="Y")                    # data encoding
    for each layer l:                                   # trainable block
        RY(w[l, q, 0]) RZ(w[l, q, 1])  on every qubit q
        CZ ring: CZ(q, q+1 mod n) for q = 0..n-1
    readout  f(x) = <Z_0>              (local cost)
          or f(x) = <Z_0 Z_1 ... Z_n-1> (global cost)
    classical head  p(y=1|x) = sigmoid(a * f(x) + b)

Weights have shape (n_layers, n_qubits, 2).

Why a CZ ring (rather than CNOT): CZ is diagonal in the computational basis,
so it commutes with every Z-type readout. A new layer whose rotation angles
are zero therefore leaves f(x) exactly unchanged when it is appended at the
end of the circuit, which is what makes "near-identity" layer insertion in
layer-wise learning well defined. CZ and CNOT differ only by Hadamards on the
target, so the entangling power of the ring is the same. Skolik et al. (2021)
also used CZ entanglers.
"""
from __future__ import annotations

import pennylane as qml


def entangler(n_qubits: int):
    if n_qubits == 1:
        return
    pairs = [(q, (q + 1) % n_qubits) for q in range(n_qubits)]
    if n_qubits == 2:
        pairs = pairs[:1]
    for a, b in pairs:
        qml.CZ(wires=[a, b])


def observable(n_qubits: int, cost: str):
    if cost == "local":
        return qml.PauliZ(0)
    if cost == "global":
        return qml.prod(*[qml.PauliZ(q) for q in range(n_qubits)])
    raise ValueError(cost)


def make_qnode(n_qubits: int, n_layers: int, cost: str = "local",
               device: str = "default.qubit", diff_method: str = "backprop",
               shots=None, interface: str = "torch"):
    """Return a PennyLane QNode f(x, weights) -> expectation value (broadcast over a batch)."""
    dev = qml.device(device, wires=n_qubits)

    def circuit(x, weights):
        qml.AngleEmbedding(x, wires=range(n_qubits), rotation="Y")     # data encoding
        for l in range(n_layers):                                       # trainable layers
            for q in range(n_qubits):
                qml.RY(weights[..., l, q, 0], wires=q)
                qml.RZ(weights[..., l, q, 1], wires=q)
            entangler(n_qubits)
        return qml.expval(observable(n_qubits, cost))                   # readout

    qnode = qml.QNode(circuit, dev, interface=interface, diff_method=diff_method)
    if shots is not None:
        qnode = qml.set_shots(qnode, shots=shots)
    return qnode
```

- Lines 1-21: the description of the architecture, and why CZ is used: CZ is diagonal, so it commutes with the Z readout, which makes a zero-angle layer exactly the identity for the output.
- Lines 27-34 `entangler`: applies CZ to the pairs (0,1), (1,2), ..., (9,0): the ring. (Special cases for 1 or 2 qubits.)
- Lines 37-42 `observable`: what we measure. "local" = Z on qubit 0; "global" = the product of Z on all qubits (used only in the barren-plateau diagnostic).
- Lines 45-63 `make_qnode`: builds the circuit: - line 52: `AngleEmbedding(x, rotation="Y")` = RY(x_q) on each qubit q (the encoding); - lines 53-57: for each layer, RY and RZ on every qubit with the trainable angles, then the CZ ring; - line 58: return the expectation value $\langle Z_0 \rangle$; - `diff_method` chooses how gradients are computed ("backprop" in simulation, or "parameter-shift" as on hardware); `shots` optionally makes it sample like a real device.

This is the **reference definition** of the model: the notebook runs this PennyLane circuit and the fast simulator side by side and shows they give identical outputs.

> **Say this to sir:** "circuit.py defines the model in PennyLane: AngleEmbedding with RY, then per layer RY and RZ on each qubit and a CZ ring, and the expectation of Z on qubit 0. It is the reference model we verify the fast simulator against."

18. fastsim.py: the fast simulator

This is the most technical file. You do not need to understand every line; understand **what** it does and **how we know it is right**.

18.1 What a simulator does

A state of 10 qubits is a list of 1024 complex numbers ψ (one "amplitude" per basis state |0000000000⟩ ... |1111111111⟩). The probability of a basis state is |amplitude|². Each gate is a mathematical operation on this list. $\langle Z_o \rangle$ is the sum over all basis states of probability × (+1 if qubit 0 is 0, -1 if it is 1).

PennyLane does this in a general way that works for any circuit. Our simulator is specialised for our circuit and processes many images at once (a whole batch in one matrix operation), which makes it more than 10 times faster.

18.2 Setting up

src/fastsim.py (lines 1-48)

```
"""Vectorised PyTorch simulator of exactly the circuit defined in circuit.py.

PennyLane (src/circuit.py) is the reference definition of the model. Running
tens of thousands of training updates through PennyLane's generic simulator
is more than 10x too slow for the required number of seeds, so the experiments execute
the *same* circuit with this specialised simulator. It was checked against
PennyLane on real MNIST inputs (expectation values, gradients, parameter-shift
values and finite-shot distributions all agree); the notebook repeats the
expectation-value check.

Conventions: wire 0 is the most significant bit of the basis index (PennyLane
convention). Weights have shape (L, n, 2): [..., 0] -> RY angle, [..., 1] -> RZ.
"""
from __future__ import annotations

import numpy as np
import torch

CDT = torch.complex128
RDT = torch.float64


class Engine:
    def __init__(self, n_qubits: int, cost: str = "local"):
        self.n = n = n_qubits
        self.dim = 2 ** n
        idx = torch.arange(self.dim)
        # bits[z, q] = value of qubit q in basis state z (q=0 most significant)
        self.bits = torch.stack([(idx >> (n - 1 - q)) & 1 for q in range(n)], dim=1).to(RDT)
        self.spin = 1.0 - 2.0 * self.bits                    # +1 for |0>, -1 for |1>
        if n == 1:
            pairs = []
        elif n == 2:
            pairs = [(0, 1)]
        else:
            pairs = [(q, (q + 1) % n) for q in range(n)]
        cz = torch.ones(self.dim, dtype=RDT)
        for a, b in pairs:
            cz = cz * (1.0 - 2.0 * self.bits[:, a] * self.bits[:, b])
        self.cz_sign = cz
        self.cost = cost
        if cost == "local":
            self.obs = self.spin[:, 0].clone()
        elif cost == "global":
            self.obs = self.spin.prod(dim=1)
```

```
46          else:
47              raise ValueError(cost)
48
```

- Lines 1-16: description: this simulates exactly the circuit of circuit.py and was verified against PennyLane.
- Line 29: `bits[z, q]` = value (0 or 1) of qubit q in basis state number z.
- Line 30: `spin` = +1 for 0, -1 for 1 (the Z value of each qubit in each basis state).
- Lines 31-40: the CZ ring is pre-computed as a list of signs: CZ multiplies an amplitude by -1 when both qubits of a pair are 1.
- Lines 41-47: the observable: Z of qubit 0 (local) or the product of all Z (global).

18.3 Encoding, layers and the output

src/fastsim.py (lines 49-106)

```
49      # ------------------------------------------------------------------ pure
50      def embed(self, x):
51          """AngleEmbedding(rotation='Y') of x (N, n) applied to |0...0>. Real product state."""
52          x = torch.as_tensor(x, dtype=RDT)
53          c, s = torch.cos(x / 2), torch.sin(x / 2)
54          psi = torch.stack([c[:, 0], s[:, 0]], dim=1)
55          for q in range(1, self.n):
56              v = torch.stack([c[:, q], s[:, q]], dim=1)
57              psi = (psi[:, :, None] * v[:, None, :]).reshape(x.shape[0], -1)
58          return psi.to(CDT)
59
60      def apply_y(self, psi, q):
61          N = psi.shape[0]
62          p = psi.reshape(-1, 2 ** q, 2, 2 ** (self.n - q - 1))
63          a, b = p[:, :, 0, :], p[:, :, 1, :]
64          return torch.stack([-1j * b, 1j * a], dim=2).reshape(N, -1)
65
66      def diag(self, wz):
67          """Diagonal of (prod_q RZ_q(wz[q])) followed by the CZ ring. wz: (n,)."""
68          phase = -0.5 * (self.spin @ wz)                          # (dim,)
69          return torch.polar(self.cz_sign.abs(), phase) * self.cz_sign
70
71      @staticmethod
72      def kron_ry(thetas):
73          """Kronecker product of RY(theta_q) matrices (real), qubit order as given."""
74          K = torch.ones(1, 1, dtype=RDT)
75          for t in thetas:
76              c, s = torch.cos(t / 2), torch.sin(t / 2)
77              K = torch.kron(K, torch.stack([torch.stack([c, -s]), torch.stack([s, c])]))
78          return K
79
80      def layer(self, psi, w, return_mid=False):
81          """One trainable layer. All RY gates of the layer equal K_A (x) K_B
82          (first / second half of the qubits), applied as two matrix products."""
83          n, N = self.n, psi.shape[0]
84          nA = n // 2
85          KA = self.kron_ry(w[:nA, 0]).to(CDT)
86          KB = self.kron_ry(w[nA:, 0]).to(CDT)
87          p = psi.reshape(N, 2 ** nA, 2 ** (n - nA))
88          p = torch.matmul(torch.matmul(KA, p), KB.transpose(0, 1))
89          psi = p.reshape(N, -1)
90          mid = psi
91          psi = psi * self.diag(w[:, 1])
92          return (psi, mid) if return_mid else psi
93
94      def state(self, x, weights):
95          psi = self.embed(x)
96          for l in range(weights.shape[0]):
97              psi = self.layer(psi, weights[l])
98          return psi
99
100     def expval(self, x, weights):
```

```
101         psi = self.state(x, weights)
102         return (psi.real ** 2 + psi.imag ** 2) @ self.obs
103
104     def expval_chunked(self, x, weights, chunk=2048):
105         x = torch.as_tensor(x, dtype=RDT)
106         return torch.cat([self.expval(x[i:i + chunk], weights) for i in range(0, x.shape[0], chunk)])
```

- `embed` (lines 50-58): RY(x_q) on $|0\rangle$ gives $\cos(x/2)|0\rangle + \sin(x/2)|1\rangle$ for each qubit; the 10-qubit state is the product (Kronecker product) of the 10 one-qubit states.
- `apply_y` (lines 60-64): applies the Pauli-Y operator on one qubit (needed only for the parameter-shift trick below).
- `diag` (lines 66-70): RZ gates and the CZ ring are both "diagonal" (they only multiply each amplitude by a number), so they are combined into one list of factors.
- `kron_ry` (lines 72-78): builds the matrix of RY gates on several qubits at once.
- `layer` (lines 80-92): one trainable layer: RY gates on all qubits, done as two matrix multiplications (first 5 qubits, last 5 qubits), then the RZ+CZ diagonal.
- `state` (lines 94-98): encoding, then all layers one after another.
- `expval` (lines 100-102): $\langle Z_o \rangle = \Sigma\ |\text{amplitude}|^2 \times \text{observable value}$.
- `expval_chunked` (lines 104-106): the same for very many images, in chunks to save memory.

Because it is written in **PyTorch**, the gradient of the output with respect to all angles can be computed automatically ("autograd"); this is how exact (noiseless) gradients are obtained.

18.4 Parameter-shift values and shot sampling

src/fastsim.py (lines 108-164)

```
108     # ------------------------------------------------- parameter-shift values
109     def shift_values(self, x, weights, trainable_layers):
110         """Exact values f(theta +- pi/2 e_k) for every parameter k in `trainable_layers`.
111
112         Uses R(theta +- pi/2) = (I -+ iG) R(theta)/sqrt(2) (G = Y or Z), so the
113         two shifted final states are (Psi -+ i chi_k)/sqrt(2), where chi_k is the
114         final state with the generator G inserted after gate k. Then
115             f_+-  = (<Psi|O|Psi> + <chi|O|chi>)/2  +-  Im<Psi|O|chi>.
116         Returns f0 (N,), fplus, fminus with shape (N, len(layers), n, 2).
117         No approximation is involved; values equal PennyLane's shifted tapes.
118         """
119         with torch.no_grad():
120             weights = torch.as_tensor(weights, dtype=RDT)
121             L = weights.shape[0]
122             N = x.shape[0]
123             psi = self.embed(x)
124             mids, posts = [], []
125             for l in range(L):
126                 psi, mid = self.layer(psi, weights[l], return_mid=True)
127                 mids.append(mid)
128                 posts.append(psi)
129             Psi = psi
130             obsPsi = Psi * self.obs
131             f0 = (Psi.real ** 2 + Psi.imag ** 2) @ self.obs
132             layers = list(trainable_layers)
133             fp = torch.zeros(N, len(layers), self.n, 2, dtype=RDT)
134             fm = torch.zeros_like(fp)
135             for j, l in enumerate(layers):
136                 d = self.diag(weights[l, :, 1])
137                 chis = []
138                 for q in range(self.n):              # Y inserted after RY_q
139                     chis.append(self.apply_y(mids[l], q) * d)
140                 for q in range(self.n):              # Z inserted after RZ_q (commutes with diag)
141                     chis.append(posts[l] * self.spin[:, q])
142                 chi = torch.cat(chis, dim=0)          # (2n*N, dim)
143                 for l2 in range(l + 1, L):
```

```
144                     chi = self.layer(chi, weights[l2])
145                 chi = chi.reshape(2 * self.n, N, self.dim)
146                 B = (chi.real ** 2 + chi.imag ** 2) @ self.obs           # (2n, N)
147                 im = (obsPsi.conj()[None] * chi).sum(-1).imag            # (2n, N)
148                 base = 0.5 * (f0[None] + B)
149                 plus, minus = base + im, base - im
150                 fp[:, j, :, 0], fp[:, j, :, 1] = plus[: self.n].T, plus[self.n:].T
151                 fm[:, j, :, 0], fm[:, j, :, 1] = minus[: self.n].T, minus[self.n:].T
152             return f0, fp, fm
153
154 def sample_expval(f, shots, rng):
155     """Finite-shot estimate of a +-1-valued observable with exact mean f.
156
157     Measuring S shots of an observable with eigenvalues +-1 yields k outcomes
158     +1 with k ~ Binomial(S, (1+f)/2); the estimator is 2k/S - 1. This is the
159     exact sampling distribution of a shot-based device for Z_0 or Z_0...Z_n-1.
160     """
161     f = np.asarray(f, dtype=np.float64)
162     p = np.clip((1.0 + f) / 2.0, 0.0, 1.0)
163     k = rng.binomial(shots, p)
164     return 2.0 * k / shots - 1.0
```

- `shift_values` (lines 109-152): computes f(θ + π/2) and f(θ - π/2) for **every** trainable angle at once, exactly. A clever identity (line 112-115) avoids re-running the whole circuit 640 times: R(θ ± π/2) = (I iG)R(θ)/√2, so the shifted outputs can be built from the normal state and one extra "inserted generator" state per angle. The result equals what PennyLane's own parameter-shift produces (verified: difference 10^{-15}).
- `sample_expval` (lines 154-164): **shot noise**. For an exact value f and S shots, the number k of "+1" outcomes is drawn from Binomial(S, (1+f)/2), and the estimate is 2k/S - 1. This is exactly the measurement statistics of a real device.

> **Say this to sir:** "fastsim.py is a vectorised PyTorch simulator of exactly our circuit. It gives exact expectation values and autograd gradients, exact parameter-shift values for every angle, and binomial shot sampling. We verified it against PennyLane to about 10^-16."

19. train.py: the two training schemes

This is the heart of the project.

19.1 The schedule and the starting angles

src/train.py (lines 1-73)

```
"""Direct training and layer-wise training (LL) of the quantum classifier.

Methods (L = 16 layers, s = 20 updates per stage, T = L*s = 320 updates)
  direct  all L layers present and trainable from the start, angles ~ U(-pi, pi);
          T updates.
  ll      start with 1 layer (identical to Direct's first layer for the same seed),
          train s updates, append a near-identity layer (angles ~ U(-0.1, 0.1)),
          train s updates, ... until L layers. All present layers keep training.
          Same total budget T as Direct.

Gradient modes
  analytic  exact gradients (noiseless simulation)
  shots     parameter-shift gradients in which every circuit expectation value is
            estimated from a finite number of measurement shots (sampling noise)

The classical head p = sigmoid(a * <Z0> + b) is always trainable.
"""
from __future__ import annotations

import math
import platform
import time
from datetime import datetime, timezone

import numpy as np
import torch

from .data import make_split
from .fastsim import Engine, sample_expval

EPS = 1e-12


def versions():
    import pennylane, sklearn, scipy
    return {"python": platform.python_version(), "numpy": np.__version__,
            "torch": torch.__version__, "pennylane": pennylane.__version__,
            "scikit-learn": sklearn.__version__, "scipy": scipy.__version__,
            "platform": platform.platform()}


def is_ll(method):
    return method in ("ll", "ll_joint")       # "ll_joint" = name used in the original run files


def schedule(method, L, s):
    """List of training stages (depth, trainable_layers, n_updates)."""
    if method == "direct":
        return [(L, list(range(L)), L * s)]
    if is_ll(method):
        return [(d, list(range(d)), s) for d in range(1, L + 1)]
    raise ValueError(method)


def init_weights(method, L, n, seed, init_range=math.pi, ident_range=0.1):
    """Direct: every layer U(-pi, pi). LL: layer 0 identical to Direct's layer 0
    for the same seed (paired comparison); later layers U(-0.1, 0.1)."""
    w_rand = np.random.default_rng([seed, 1]).uniform(-init_range, init_range, (L, n, 2))
    if method == "direct":
        return w_rand
    w = np.random.default_rng([seed, 2]).uniform(-ident_range, ident_range, (L, n, 2))
    w[0] = w_rand[0]
```

```
63      return w
64
65
66  def bce(p, y):
67      p = np.clip(p, EPS, 1 - EPS)
68      return float(-np.mean(y * np.log(p) + (1 - y) * np.log(1 - p)))
69
70
71  def sigmoid(z):
72      return 1.0 / (1.0 + np.exp(-z))
73
```

- Lines 1-17: description of both methods (budget T = 16 × 20 = 320 updates) and of the two gradient modes ("analytic" = exact, "shots" = parameter shift with sampled values).
- `is_ll` (lines 42-43): recognises LL runs (old result files call the method "ll_joint").
- `schedule` (lines 46-52): the core difference between the methods, as a list of stages (depth, which layers train, how many updates): - Direct: one stage: `(16, [0..15], 320)`; - LL: 16 stages: `(1, [0], 20)`, `(2, [0, 1], 20)`, ..., `(16, [0..15], 20)`.
- `init_weights` (lines 55-63): starting angles. Direct: every angle uniform in [-π, π]. LL: first layer = exactly Direct's first layer for the same seed; all other layers uniform in [-0.1, 0.1] (near identity).
- `bce` and `sigmoid` (lines 66-72): the loss and the squashing function from Chapter 2. `EPS` avoids log(0).

19.2 Adam that only updates the active layers

src/train.py (lines 75-97)

```
75  class MaskedAdam:
76      """Adam in which only the currently trainable entries are updated.
77
78      Each entry keeps its own step counter, so a layer that starts training late
79      gets fresh, correctly bias-corrected moments.
80      """
81
82      def __init__(self, shapes, lr, b1=0.9, b2=0.999, eps=1e-8):
83          self.lr, self.b1, self.b2, self.eps = lr, b1, b2, eps
84          self.m = [np.zeros(s) for s in shapes]
85          self.v = [np.zeros(s) for s in shapes]
86          self.t = [np.zeros(s) for s in shapes]
87
88      def step(self, params, grads, masks):
89          for i, (p, g, mk) in enumerate(zip(params, grads, masks)):
90              mk = np.asarray(mk, dtype=bool)
91              self.t[i][mk] += 1
92              self.m[i][mk] = self.b1 * self.m[i][mk] + (1 - self.b1) * g[mk]
93              self.v[i][mk] = self.b2 * self.v[i][mk] + (1 - self.b2) * g[mk] ** 2
94              mh = self.m[i][mk] / (1 - self.b1 ** self.t[i][mk])
95              vh = self.v[i][mk] / (1 - self.b2 ** self.t[i][mk])
96              p[mk] -= self.lr * mh / (np.sqrt(vh) + self.eps)
97
```

`MaskedAdam` is the Adam optimiser with a **mask**: a True/False array saying which angles are currently trainable. Only those are updated. Each angle has its own step counter `t`, so a layer that joins late starts with fresh, correctly scaled Adam statistics.

- `m` = running average of gradients (momentum), `v` = running average of squared gradients.
- Lines 94-95: the standard Adam bias correction; line 96: the update `p -= lr × mˆ / (√vˆ + eps)`.

19.3 The model and its gradients

src/train.py (lines 99-152)

```
 99  class Model:
100      def __init__(self, n_qubits, n_layers, cost, weights):
101          self.eng = Engine(n_qubits, cost)
102          self.w = np.array(weights, dtype=np.float64)
103          self.a = np.array([1.0])
104          self.b = np.array([0.0])
105
106      def expval(self, x, depth):
107          with torch.no_grad():
108              return self.eng.expval_chunked(x, torch.as_tensor(self.w[:depth])).numpy()
109
110      def metrics(self, y, f):
111          p = sigmoid(self.a[0] * f + self.b[0])
112          return bce(p, y), float(np.mean((p >= 0.5) == (y >= 0.5)))
113
114      def grads_analytic(self, x, y, depth):
115          """Exact gradient of the minibatch loss (automatic differentiation)."""
116          w = torch.tensor(self.w[:depth], requires_grad=True)
117          a = torch.tensor(self.a[0], requires_grad=True)
118          b = torch.tensor(self.b[0], requires_grad=True)
119          z = a * self.eng.expval(torch.as_tensor(x), w) + b
120          loss = torch.nn.functional.binary_cross_entropy_with_logits(z, torch.as_tensor(y))
121          gw, ga, gb = torch.autograd.grad(loss, (w, a, b))
122          g = np.zeros_like(self.w)
123          g[:depth] = gw.numpy()
124          return loss.item(), g, np.array([ga.item()]), np.array([gb.item()])
125
126      def grads_shots(self, x, y, depth, trainable, shots, rng):
127          """Parameter-shift gradient with every expectation estimated from `shots` shots.
128
129          The forward value f_i and each shifted value f_i(theta +- pi/2 e_k) are
130          independent S-shot estimates; the classical chain rule then uses them
131          exactly as a shot-based quantum computer + classical optimiser would.
132          """
133          w = torch.as_tensor(self.w[:depth])
134          f0, fp, fm = self.eng.shift_values(torch.as_tensor(x), w, trainable)
135          f_hat = sample_expval(f0.numpy(), shots, rng)
136          fp_hat = sample_expval(fp.numpy(), shots, rng)
137          fm_hat = sample_expval(fm.numpy(), shots, rng)
138          df = 0.5 * (fp_hat - fm_hat)                          # parameter-shift rule
139          p = sigmoid(self.a[0] * f_hat + self.b[0])
140          loss = bce(p, y)
141          dz = (p - y) / len(y)                                 # d loss / d z
142          g = np.zeros_like(self.w)
143          g[trainable] = np.einsum("i,ijkl->jkl", dz * self.a[0], df)
144          return loss, g, np.array([np.sum(dz * f_hat)]), np.array([np.sum(dz)])
145
146
147  def circuit_evals(n_trainable, batch, mode):
148      """Number of circuit executions needed for one gradient estimate."""
149      if mode == "shots":
150          return int(batch * (1 + 2 * int(n_trainable)))
151      return int(batch)
152
```

- `Model.__init__`: holds the simulator `eng`, the angles `w`, and the classical head a = 1, b = 0 at the start.
- `expval(x, depth)`: $\langle Z_0 \rangle$ for images x using only the first `depth` layers (this is how LL "has" fewer layers: layers beyond the current depth are simply not applied).
- `metrics`: loss and accuracy from $\langle Z_0 \rangle$ values.
- `grads_analytic` (lines 114-124): the noiseless gradient. PyTorch computes the loss for the 16-image batch and differentiates it automatically with respect to the angles, a and b.
- `grads_shots` (lines 126-144): the noisy, hardware-like gradient: 1. get the exact normal and shifted values (line 134); 2. replace each by a **shot estimate** (lines 135-137); 3. parameter-shift rule `df = (f+`

- `f-)/2` (line 138); 4. compute the prediction and loss from the *noisy* $\langle Z_o \rangle$ (lines 139-140); 5. chain rule: d loss/d θ = Σ over images of (p - y)/N × a × df (lines 141-143). This is exactly how a classical optimiser would use measurements from a quantum computer.
- `circuit_evals` (lines 147-151): counts circuit executions: 16 × (1 + 2 × trainable angles) per update with shots.

19.4 The training loop

src/train.py (lines 154-212)

```
154  def train_run(cfg: dict) -> dict:
155      """Run one training job described by cfg; return the full result record."""
156      t_start = time.time()
157      n, L = cfg["n_qubits"], cfg["n_layers"]
158      split = make_split(tuple(cfg["digits"]), n_features=n, n_train=cfg["n_train"],
159                         n_val=cfg["n_val"], n_test=cfg["n_test"])
160      stages = schedule(cfg["method"], L, cfg["updates_per_stage"])
161      model = Model(n, L, cfg["cost"],
162                    init_weights(cfg["method"], L, n, cfg["seed"], cfg["init_range"], cfg["ident_range"]))
163      opt = MaskedAdam([model.w.shape, (1,), (1,)], cfg["lr"])
164      mode = cfg.get("mode", "analytic")
165      batch_rng = np.random.default_rng([cfg["seed"], 3])
166      shot_rng = np.random.default_rng([cfg["seed"], 4, int(cfg.get("shots") or 0)])
167      B = cfg["batch_size"]
168      xtr, ytr = split.x_train, split.y_train
169      order, ptr = batch_rng.permutation(len(ytr)), 0
170
171      H = {k: [] for k in ("iter", "depth", "train_loss", "train_acc", "val_loss", "val_acc",
172                           "test_loss", "test_acc", "batch_loss", "grad_norm_trainable",
173                           "grad_norm_active", "grad_mean_abs_trainable", "circuit_evals")}
174      stage_starts = []
175      x_all = np.concatenate([xtr, split.x_val, split.x_test])
176      n1, n2 = len(ytr), len(ytr) + len(split.y_val)
177
178      def record(it, depth, batch_loss, gn_tr, gn_act, gmean, cev):
179          f = model.expval(x_all, depth)          # evaluate train + validation + test
180          trl, tra = model.metrics(ytr, f[:n1])
181          vl, va = model.metrics(split.y_val, f[n1:n2])
182          tel, tea = model.metrics(split.y_test, f[n2:])
183          for k, v in zip(H.keys(), (it, depth, trl, tra, vl, va, tel, tea, batch_loss,
184                                     gn_tr, gn_act, gmean, cev)):
185              H[k].append(v)
186
187      record(0, stages[0][0], None, None, None, None, 0)
188      it, total_cev = 0, 0
189      for depth, trainable, n_up in stages:
190          stage_starts.append({"iter": it, "depth": depth, "trainable_layers": trainable,
191                               "updates": n_up})
192          mask_w = np.zeros(model.w.shape, dtype=bool)
193          mask_w[trainable] = True
194          for _ in range(n_up):
195              if ptr + B > len(ytr):
196                  order, ptr = batch_rng.permutation(len(ytr)), 0
197              idx = order[ptr:ptr + B]
198              ptr += B
199              if mode == "shots":
200                  loss, g, ga, gb = model.grads_shots(xtr[idx], ytr[idx], depth, trainable,
201                                                      cfg["shots"], shot_rng)
202                  gn_act = None
203              else:
204                  loss, g, ga, gb = model.grads_analytic(xtr[idx], ytr[idx], depth)
205                  gn_act = float(np.linalg.norm(g[:depth]))
206              gtr = g[mask_w]
207              total_cev += circuit_evals(mask_w.sum(), B, mode)
208              opt.step([model.w, model.a, model.b], [g, ga, gb],
209                       [mask_w, np.ones(1, bool), np.ones(1, bool)])
210              it += 1
211              record(it, depth, loss, float(np.linalg.norm(gtr)), gn_act,
212                     float(np.mean(np.abs(gtr))), total_cev)
```

Step by step:

1. Lines 157-162: get the data split, the schedule for this method, and the starting angles.
2. Line 163: the masked Adam optimiser for the angles, a and b.
3. Lines 165-166: two random generators from the seed: one for the order of mini-batches, one for shot noise.
4. Lines 171-185: `record` evaluates the current model **exactly** on all train, validation and test images and appends loss and accuracy to the history `H`. It is called before training (update 0) and after every update, so we get a full curve of 321 points.
5. Lines 189-193: for each stage: build the mask of trainable layers.
6. Lines 194-198: take the next 16 images (reshuffle when the training set is used up).
7. Lines 199-205: compute the gradient: with shots (noisy parameter shift) or exactly.
8. Line 208: Adam update of the trainable angles and of a, b.
9. Lines 211-212: record the metrics plus the gradient size (`grad_norm`, and the mean |gradient| per trainable angle, used in the analysis).

For LL, `depth` grows from 1 to 16 as the stages pass; for Direct it is always 16.

19.5 The summary of one run

src/train.py (lines 214-239)

```
214     thr = cfg["loss_threshold"]
215     final = {k: H[k][-1] for k in ("train_loss", "train_acc", "val_loss", "val_acc",
216                                    "test_loss", "test_acc")}
217     summary = {
218         **{"final_" + k: v for k, v in final.items()},
219         "success": bool(final["train_loss"] < thr and final["test_acc"] >= cfg["acc_target"]),
220         "iter_converge_loss": next((i for i, l in zip(H["iter"], H["train_loss"]) if l < thr), None),
221         "iter_reach_test_acc": next((i for i, a in zip(H["iter"], H["test_acc"])
222                                      if a >= cfg["acc_target"]), None),
223         "total_updates": it,
224         "final_depth_updates": it - stage_starts[-1]["iter"],
225         "total_circuit_evals": int(total_cev),
226         "wall_time_s": time.time() - t_start,
227     }
228     return {
229         "config": cfg,
230         "data_info": split.info,
231         "versions": versions(),
232         "timestamp_utc": datetime.now(timezone.utc).isoformat(),
233         "stages": stage_starts,
234         "history": H,
235         "summary": summary,
236         "final_params": {"weights": model.w.tolist(), "scale": float(model.a[0]),
237                          "bias": float(model.b[0])},
238         "complete": True,
239     }
```

- Lines 215-216: the final values after update 320.
- Line 219: **success** = final training loss < 0.3466 and final test accuracy ≥ 0.90.
- Lines 221-222: **iterations to converge** = first update with test accuracy ≥ 0.90 (None if never).
- Lines 228-239: everything is returned and saved: settings, data info, library versions, time stamp, stages, the full history, the summary and the **final trained parameters** (so any model can be re-checked later).

Say this to sir: "train.py implements both schemes through one schedule: Direct is one stage of 320 updates at depth 16; LL is 16 stages of 20 updates with depth growing by one. A masked Adam only updates the active layers. Gradients are exact or hardware-like parameter-shift with shot-sampled values. After every update the model is evaluated, and success and iterations-to-90% are computed from that history."

20. runner.py and run_all.py: running the experiment

20.1 runner.py

src/runner.py (lines 1-70)

```
"""Building, running and saving training jobs (in parallel, resumable)."""
from __future__ import annotations

import json
import os
import time
import traceback
from pathlib import Path

from . import config as C


def make_cfg(experiment, method, seed, shots=None):
    """experiment: 'noiseless', 'shots1000' or 'shots500'."""
    cfg = dict(**C.TASK, **C.DATA, **C.TRAIN, method=method, seed=seed,
               mode="shots" if shots else "analytic", shots=shots,
               loss_threshold=C.LOSS_THRESHOLD, acc_target=C.ACC_TARGET)
    cfg["digits"] = list(cfg["digits"])
    return experiment, cfg


def run_path(experiment, cfg) -> Path:
    return C.RESULTS / "runs" / experiment / cfg["method"] / f"seed_{cfg['seed']:03d}.json"


def is_done(path: Path) -> bool:
    try:
        return bool(json.loads(path.read_text()).get("complete"))
    except Exception:
        return False


def execute(task):
    """Worker: run one job and save it (write to .tmp, then rename = never half-written)."""
    experiment, cfg = task
    import torch
    torch.set_num_threads(1)
    from .train import train_run

    path = run_path(experiment, cfg)
    if is_done(path):
        return f"skip {path.relative_to(C.ROOT)}"
    path.parent.mkdir(parents=True, exist_ok=True)
    t = time.time()
    try:
        rec = train_run(cfg)
        rec["experiment"] = experiment
        tmp = path.with_suffix(".tmp")
        tmp.write_text(json.dumps(rec))
        os.replace(tmp, path)
        return f"done {path.relative_to(C.ROOT)} {time.time() - t:.0f}s"
    except Exception:
        path.with_suffix(".error.txt").write_text(traceback.format_exc())
        return f"FAIL {path.relative_to(C.ROOT)}"


def run_tasks(tasks, workers, log):
    """Run jobs on `workers` CPU cores; finished jobs are skipped (resumable)."""
    from multiprocessing import Pool

    todo = [t for t in tasks if not is_done(run_path(*t))]
    log(f"{len(tasks)} jobs, {len(tasks) - len(todo)} already done, {len(todo)} to run")
    fails, t0 = 0, time.time()
    if todo:
```

```
65          with Pool(workers, maxtasksperchild=20) as pool:
66              for i, msg in enumerate(pool.imap_unordered(execute, todo), 1):
67                  fails += msg.startswith("FAIL")
68                  el = time.time() - t0
69                  log(f"[{i}/{len(todo)}] {msg} | {el / 60:.1f} min, eta {el / i * (len(todo) - i) / 60:.1f} min")
70      return fails
```

- `make_cfg` (lines 13-19): builds the complete settings dictionary for one run (task, data, training settings, method, seed, shots).
- `run_path` (lines 22-23): where the run is saved, e.g. `results/runs/shots500/ll/seed_007.json`.
- `is_done` (lines 26-30): a run is skipped if its file already exists and is marked complete. This makes the whole experiment **resumable**: if the laptop restarts, nothing finished is lost.
- `execute` (lines 33-54): runs one job and saves it safely: it first writes to a temporary file and then renames it, so a crash can never leave a half-written result. Errors are saved to an `.error.txt` file.
- `run_tasks` (lines 57-70): runs all unfinished jobs in parallel on all CPU cores and prints progress with an estimated time remaining.

20.2 run_all.py

run_all.py (lines 1-56)

```
1   """Run the experiments and make the figures.
2
3   Usage (from the project folder, with the venv's python):
4       python run_all.py figures      # summary table + all figures from the saved results (~1 min)
5       python run_all.py train        # Direct and LL, 30 seeds: noiseless, 1000 shots, 500 shots (~5 h)
6       python run_all.py diagnostic   # barren-plateau gradient-variance diagnostic (~1 min)
7   Training skips every run whose result file already exists, so with the saved
8   results in results/ only `figures` does any work.
9   Options: --workers N (default: number of CPU cores)
10  """
11  from __future__ import annotations
12
13  import argparse
14  import os
15  import sys
16  import time
17  from pathlib import Path
18
19  ROOT = Path(__file__).resolve().parent
20  sys.path.insert(0, str(ROOT))
21
22  from src import config as C  # noqa: E402
23
24
25  def log(msg):
26      print(f"{time.strftime('%H:%M:%S')} {msg}", flush=True)
27
28
29  def train_tasks():
30      from src.runner import make_cfg
31      tasks = []
32      for experiment, shots in (("noiseless", None), ("shots1000", 1000), ("shots500", 500)):
33          for m in C.METHODS:
34              for s in C.SEEDS:
35                  tasks.append(make_cfg(experiment, m, s, shots=shots))
36      return tasks
37
38
39  def main():
40      ap = argparse.ArgumentParser()
41      ap.add_argument("stage", choices=["train", "diagnostic", "figures"])
42      ap.add_argument("--workers", type=int, default=os.cpu_count())
43      a = ap.parse_args()
44      if a.stage == "train":
45          from src.runner import run_tasks
```

```
46          sys.exit(1 if run_tasks(train_tasks(), a.workers, log) else 0)
47      if a.stage == "diagnostic":
48          from src.gradients import run_diagnostic
49          run_diagnostic(log, a.workers)
50      if a.stage == "figures":
51          from src.figures import make_all
52          make_all(log)
53
54
55  if __name__ == "__main__":
56      main()
```

The single entry point. `train_tasks` (lines 29-36) creates the 180 jobs: 3 settings × 2 methods × 30 seeds. `main` reads which step you asked for:

- `python run_all.py train`: run all training jobs (skips finished ones; with the saved results nothing is retrained);
- `python run_all.py diagnostic`: barren-plateau diagnostic;
- `python run_all.py figures`: the table and all figures from the saved results (about 1 minute).

> **Say this to sir:** "runner.py turns each setting-method-seed combination into a job, runs jobs in parallel, saves each safely as JSON and skips finished ones, so the experiment is resumable. run_all.py has three commands: train, diagnostic and figures."

21. gradients.py: the barren-plateau diagnostic

src/gradients.py (lines 1-99)

```
"""Barren-plateau diagnostic: gradient variance over random initialisations.

Cost per circuit: C(theta) = mean over a fixed batch of 16 real MNIST training
images (digits of the hard config) of <O>, with O = Z_0 (local) or
Z_0 Z_1 ... Z_{n-1} (global). For each (n_qubits, depth, cost) we draw
N_INIT = 100 parameter vectors theta ~ U(-pi, pi) and compute the exact
gradient. Reported:
  var_fixed  Var_theta[dC/dtheta_(layer 0, qubit 0, RY)]  (fixed component)
  var_mean   mean over all components of Var_theta[dC/dtheta_k]
  95% bootstrap confidence intervals (2000 resamples of the 100 inits).
Additionally (local cost, depth sweep) the same quantities for the layer-wise
initialisation (layer 0 ~ U(-pi, pi), later layers ~ U(-0.1, 0.1)).
"""
from __future__ import annotations

import json
import math
from multiprocessing import Pool

import numpy as np

from . import config as C

N_INIT = 100
BATCH = 16
QUBITS = (4, 6, 8, 10, 12)
FIXED_DEPTH = C.TASK["n_layers"]
DEPTH_QUBITS = C.TASK["n_qubits"]
DEPTHS = tuple(range(1, 21))
OUT = C.RESULTS / "diagnostic"


def _point(args):
    sweep, n, L, cost, init = args
    import torch
    torch.set_num_threads(1)
    from .data import make_split
    from .fastsim import Engine

    sp = make_split(tuple(C.TASK["digits"]), n_features=n)
    x = torch.tensor(sp.x_train[:BATCH])
    eng = Engine(n, cost)
    grads = np.zeros((N_INIT, L, n, 2))
    for i in range(N_INIT):
        rng = np.random.default_rng([n, L, i, 0 if cost == "local" else 1, 0 if init == "random" else 1])
        if init == "random":
            w = rng.uniform(-math.pi, math.pi, (L, n, 2))
        else:
            w = rng.uniform(-0.1, 0.1, (L, n, 2))
            w[0] = rng.uniform(-math.pi, math.pi, (n, 2))
        wt = torch.tensor(w, requires_grad=True)
        grads[i] = torch.autograd.grad(eng.expval(x, wt).mean(), wt)[0].numpy()
    g_fixed = grads[:, 0, 0, 0]
    flat = grads.reshape(N_INIT, -1)
    boot = np.random.default_rng(12345)
    bi = boot.integers(0, N_INIT, (2000, N_INIT))
    bf = np.var(g_fixed[bi], axis=1, ddof=1)
    bm = np.array([np.var(flat[b], axis=0, ddof=1).mean() for b in bi[:500]])
    return {"sweep": sweep, "n_qubits": n, "depth": L, "cost": cost, "init": init,
            "n_init": N_INIT, "batch": BATCH,
            "var_fixed": float(np.var(g_fixed, ddof=1)),
            "var_fixed_ci95": [float(np.percentile(bf, 2.5)), float(np.percentile(bf, 97.5))],
            "var_mean": float(np.var(flat, axis=0, ddof=1).mean()),
            "var_mean_ci95": [float(np.percentile(bm, 2.5)), float(np.percentile(bm, 97.5))],
            "mean_abs_grad": float(np.abs(flat).mean()),
            "grad_fixed_samples": g_fixed.tolist()}


```

```
69  def tasks():
70      t = []
71      for cost in ("local", "global"):
72          for n in QUBITS:
73              t.append(("qubits", n, FIXED_DEPTH, cost, "random"))
74          for L in DEPTHS:
75              t.append(("depth", DEPTH_QUBITS, L, cost, "random"))
76      for L in DEPTHS:
77          t.append(("depth", DEPTH_QUBITS, L, "local", "layerwise"))
78      return t
79
80
81  def run_diagnostic(log=print, workers=8):
82      OUT.mkdir(parents=True, exist_ok=True)
83      path = OUT / "bp_diagnostic.json"
84      if path.exists():
85          log("diagnostic already complete, skipping")
86          return
87      todo = sorted(tasks(), key=lambda a: -(2 ** a[1]) * a[2])
88      rows = []
89      with Pool(workers) as pool:
90          for r in pool.imap_unordered(_point, todo):
91              rows.append(r)
92              log(f"diag {r['sweep']} n={r['n_qubits']} L={r['depth']} {r['cost']} {r['init']}: "
93                  f"var_fixed={r['var_fixed']:.3e} var_mean={r['var_mean']:.3e}")
94      rows.sort(key=lambda r: (r["sweep"], r["cost"], r["init"], r["n_qubits"], r["depth"]))
95      from .train import versions
96      path.write_text(json.dumps({"description": __doc__, "versions": versions(),
97                                  "data_digits": list(C.TASK["digits"]),
98                                  "points": rows}, indent=1))
99      log(f"diagnostic written: {path}")
```

What it measures (lines 1-14): the cost C = average of ⟨O⟩ over 16 real training images, where O is Z_0 (local) or Z on all qubits (global). For each setting it draws **100 random angle vectors** and computes the exact gradient.

- Lines 24-30: 100 initialisations, batch of 16 images; qubit numbers 4, 6, 8, 10, 12 (at 16 layers); depths 1-20 (at 10 qubits).
- `_point` (lines 33-66): for one setting: build the simulator, draw 100 random starts (random in ±π, or the layer-wise start: first layer random, the rest in ±0.1), compute the gradient for each, and then: - `var_fixed`: the variance of the gradient of one fixed angle (first layer, qubit 0, RY) across the 100 starts; - `var_mean`: the variance averaged over all angles; - 95% confidence intervals by **bootstrap** (re-sampling the 100 starts 2000 times).
- `tasks` (lines 69-78): the list of settings: qubit sweep and depth sweep for local and global cost with random starts, plus the depth sweep with layer-wise starts.
- `run_diagnostic` (lines 81-99): runs everything in parallel and writes `results/diagnostic/bp_diagnostic.json`.

Why variance? Over random starts the average gradient is 0 (it is equally likely to point either way). Its *variance* tells how big gradients typically are. Barren plateau = variance shrinking towards 0.

> **Say this to sir:** "The diagnostic draws 100 random initialisations per setting, computes exact gradients on real MNIST inputs, and reports the gradient variance with bootstrap confidence intervals, for different qubit counts and depths, comparing random and layer-wise initialisation."

22. figures.py and the notebook

22.1 figures.py: table and graphs

src/figures.py (lines 1-87)

```
"""Results table and figures for Direct vs Layer-wise (LL) training.

Reads only the saved run files in results/ (nothing is retrained) and writes
    results/summary.csv          one row per (noise setting, method)
    figures/fig1_circuit.png     the variational circuit
    figures/fig2_training_schemes.png   which layers are trained when
    figures/fig3_barren_plateau.png     gradient variance vs qubits / depth
    figures/fig4_test_accuracy_curves.png
    figures/fig5_final_accuracy.png
    figures/fig6_success_rate.png
    figures/fig7_iterations_to_converge.png
"""
from __future__ import annotations

import csv
import json
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
from matplotlib.patches import FancyBboxPatch, Rectangle  # noqa: E402

from . import config as C  # noqa: E402

METHODS = {"direct": "Direct", "ll": "Layer-wise (LL)"}
COLOR = {"direct": "#eb6834", "ll": "#2a78d6"}
SETTINGS = {"noiseless": "Noiseless\n(exact gradients)",
            "shots1000": "1000 shots\n(sampling noise)",
            "shots500": "500 shots\n(more noise)"}
INK, MUTED, GRID = "#0b0b0b", "#52514e", "#e4e3df"

plt.rcParams.update({
    "font.size": 11, "axes.titlesize": 12, "axes.titleweight": "bold",
    "axes.edgecolor": MUTED, "axes.labelcolor": INK, "xtick.color": MUTED, "ytick.color": MUTED,
    "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
    "grid.color": GRID, "grid.linewidth": 0.8, "axes.axisbelow": True,
    "legend.frameon": False, "figure.dpi": 110, "savefig.dpi": 200, "savefig.bbox": "tight",
})


# ---------------------------------------------------------------- data
def load_runs(setting, method):
    files = sorted((C.RESULTS / "runs" / setting / method).glob("seed_*.json"))
    return [json.loads(f.read_text()) for f in files]


def load_all():
    return {(s, m): load_runs(s, m) for s in SETTINGS for m in METHODS}


def ci95(a):
    a = np.asarray(a, float)
    return 1.96 * a.std(ddof=1) / np.sqrt(len(a))


def summarize(runs):
    rows = []
    for (s, m), R in runs.items():
        S = [r["summary"] for r in R]
        acc = np.array([x["final_test_acc"] for x in S])
        reach = [x["iter_reach_test_acc"] for x in S if x["iter_reach_test_acc"] is not None]
        rows.append({
```

```
            "setting": s, "method": m, "runs": len(R),
            "mean_test_acc": round(float(acc.mean()), 4), "ci95_test_acc": round(float(ci95(acc)), 4),
            "mean_final_train_loss": round(float(np.mean([x["final_train_loss"] for x in S])), 4),
            "successful_runs": int(sum(x["success"] for x in S)),
            "success_rate_pct": round(float(100 * np.mean([x["success"] for x in S])), 1),
            "runs_reaching_90pct": len(reach),
            "median_iters_to_90pct": float(np.median(reach)) if reach else None,
        })
    return rows


def write_summary(rows, path=None):
    path = path or C.RESULTS / "summary.csv"
    with open(path, "w", newline="") as fh:
        w = csv.DictWriter(fh, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)
    return path


def _row(rows, s, m):
    return next(r for r in rows if r["setting"] == s and r["method"] == m)

```

- Lines 1-12: the list of outputs.
- Lines 27-41: names, colours (orange = Direct, blue = LL) and the plot style.
- `load_runs` / `load_all` (lines 44-50): read the 30 JSON files for every (setting, method).
- `ci95` (lines 53-55): the 95% confidence interval of a mean: 1.96 × standard deviation / √n.
- `summarize` (lines 58-73): for each (setting, method): number of runs, mean test accuracy and CI, mean final training loss, number and percentage of successful runs, how many runs reached 90%, and the median iterations to 90%.
- `write_summary` (lines 76-82): saves the table as `results/summary.csv`.

src/figures.py (lines 89-170)

```
# ------------------------------------------------------------------ circuits
def _gate(ax, x, y, text, fc, w=0.9, h=0.62, fs=9):
    ax.add_patch(FancyBboxPatch((x - w / 2, y - h / 2), w, h, boxstyle="round,pad=0.02,rounding_size=0.08",
                                fc=fc, ec=INK, lw=1, zorder=3))
    ax.text(x, y, text, ha="center", va="center", fontsize=fs, zorder=4)


def fig_circuit(path):
    n = C.TASK["n_qubits"]
    fig, ax = plt.subplots(figsize=(13, 6.2))
    ax.set_axis_off()
    ys = [n - 1 - q for q in range(n)]
    for q, y in enumerate(ys):
        ax.plot([0.6, 10.6 if q == 0 else 9.9], [y, y], color=INK, lw=1, zorder=1)
        ax.text(0.45, y, rf"$q_{{{q}}}$: $|0\rangle$", ha="right", va="center", fontsize=10)
        _gate(ax, 1.5, y, rf"$R_Y(x_{{{q}}})$", "#d9efe4", w=1.2)        # data encoding
        _gate(ax, 3.4, y, r"$R_Y(\theta)$", "#dce9f8")                   # trainable rotations
        _gate(ax, 4.6, y, r"$R_Z(\theta)$", "#dce9f8")
    # CZ ring: (0,1),(2,3).. | (1,2),(3,4).. | (9,0); CZ gates commute, so this order is exact
    cols = {5.8: [(q, q + 1) for q in range(0, n - 1, 2)],
            6.6: [(q, q + 1) for q in range(1, n - 1, 2)],
            7.4: [(n - 1, 0)]}
    for x, pairs in cols.items():
        for a, b in pairs:
            ax.plot([x, x], [ys[a], ys[b]], color=INK, lw=1.2, zorder=2)
            ax.plot([x, x], [ys[a], ys[b]], "o", color=INK, ms=6, zorder=3)
    # one-layer bracket
    ax.add_patch(Rectangle((2.75, -0.55), 5.1, n + 0.1, fill=False, ec=COLOR["ll"],
                           lw=1.8, ls="--", zorder=0))
    ax.text(5.3, n - 0.15, "one trainable layer\n(repeated L = 16 times)", ha="center",
            va="bottom", fontsize=11, color=COLOR["ll"], fontweight="bold")
    ax.text(1.5, n - 0.15, "data encoding", ha="center", va="bottom", fontsize=11, fontweight="bold")
    ax.text(6.6, -0.95, "CZ ring (entangling)", ha="center", va="top", fontsize=10, color=MUTED)
    ax.text(4.0, -0.95, "20 angles per layer", ha="center", va="top", fontsize=10, color=MUTED)
```

```
    ax.text(9.1, -0.95, "... layers 2-16 ...", ha="center", va="top", fontsize=10, color=MUTED)
    for y in ys:
        ax.text(9.1, y, "...", ha="center", va="center", fontsize=12, color=MUTED,
                bbox=dict(fc="white", ec="none", pad=1), zorder=3)
    # measurement on qubit 0 and classical head
    _gate(ax, 10.6, ys[0], r"measure $\langle Z_0\rangle$", "#fbe3d6", w=1.4)
    ax.text(10.6, n - 0.15, "readout", ha="center", va="bottom", fontsize=11, fontweight="bold")
    ax.annotate("", xy=(12.9, ys[0] - 1.5), xytext=(10.9, ys[0] - 0.35),
                arrowprops=dict(arrowstyle="->", color=INK))
    ax.text(12.9, ys[0] - 1.6, r"$p(\mathrm{digit\ 9}\,|\,x) = \sigma(a\,\langle Z_0\rangle + b)$"
            "\nclassical head (a, b trained too)", ha="center", va="top", fontsize=10.5)
    ax.set_xlim(-0.6, 15.0)
    ax.set_ylim(-1.5, n + 1.2)
    ax.set_title("Quantum classifier: 10 qubits, 16 layers, MNIST digits 4 vs 9 "
                 "(10 PCA features, one per qubit)", pad=8)
    fig.savefig(path)
    plt.close(fig)


def fig_training_schemes(path):
    L, s = C.TASK["n_layers"], C.TRAIN["updates_per_stage"]
    T = L * s
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.6), sharey=True)
    for ax, m in zip(axes, METHODS):
        for l in range(1, L + 1):
            start = 0 if m == "direct" else (l - 1) * s
            new = s if (m == "ll" and l > 1) else 0
            ax.barh(l, T - start - new, left=start + new, height=0.78, color=COLOR[m])
            if new:
                ax.barh(l, new, left=start, height=0.78, color="#a9c9f0")
        ax.set_xlim(0, T)
        ax.set_ylim(0.4, L + 0.6)
        ax.set_xticks(range(0, T + 1, 40))
        ax.set_yticks([1, 4, 8, 12, 16])
        ax.set_xlabel("training update (Adam step, batch of 16 images)")
        ax.grid(axis="y", visible=False)
    axes[0].set_ylabel("circuit layer")
    axes[0].set_title("(a) Direct: all 16 layers trained from step 0\nrandom angles in [-π, π]")
    axes[1].set_title("(b) Layer-wise: start with 1 layer, add 1 layer every 20 updates\n"
                      "new layer starts near identity (angles in [-0.1, 0.1])")
    axes[1].text(6, L - 1.5, "light blue = layer just added (starts ≈ identity)\n"
                 "layers already present keep training", ha="left", va="center", fontsize=9.5,
                 color=MUTED)
    fig.suptitle("Both protocols get the same budget: 320 updates", fontsize=12, y=1.02)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)

```

The drawing of Figure 1 (the circuit, drawn gate by gate with matplotlib so it is clean and readable) and Figure 2 (horizontal bars: which layers train at which update).

src/figures.py (lines 172-220)

```
# ------------------------------------------------------------ barren plateau
def fig_barren_plateau(path):
    pts = json.loads((C.RESULTS / "diagnostic" / "bp_diagnostic.json").read_text())["points"]

    def series(**kw):
        P = sorted([p for p in pts if all(p[k] == v for k, v in kw.items())],
                   key=lambda p: (p["n_qubits"], p["depth"]))
        return P

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
    ax = axes[0]
    P = series(sweep="qubits", cost="global", init="random")
    x = [p["n_qubits"] for p in P]
    ax.plot(x, [p["var_fixed"] for p in P], "-o", color=COLOR["direct"], lw=2, ms=7)
    ax.fill_between(x, [p["var_fixed_ci95"][0] for p in P], [p["var_fixed_ci95"][1] for p in P],
                    color=COLOR["direct"], alpha=0.15, lw=0)
    ax.set_yscale("log")
    ax.set_xticks(x)
    ax.set_xlabel("number of qubits")
```

```
    ax.set_ylabel(r"Var[$\partial C / \partial\theta$]  (log scale)")
    ax.set_title("(a) Barren plateau: gradients vanish\nexponentially with qubit count "
                 "(16 layers, random init)")

    ax = axes[1]
    for init, m, lab in (("random", "direct", "random init (Direct)"),
                         ("layerwise", "ll", "layer-wise init (LL)")):
        P = series(sweep="depth", cost="local", init=init)
        x = [p["depth"] for p in P]
        ax.plot(x, [p["var_fixed"] for p in P], "-o", color=COLOR[m], lw=2, ms=5, label=lab)
        ax.fill_between(x, [p["var_fixed_ci95"][0] for p in P], [p["var_fixed_ci95"][1] for p in P],
                        color=COLOR[m], alpha=0.15, lw=0)
    r = {i: next(p for p in series(sweep="depth", cost="local", init=i) if p["depth"] == 16)["var_fixed"]
         for i in ("random", "layerwise")}
    ax.axvline(16, color=MUTED, lw=1, ls=":")
    ax.text(15.6, r["random"] * 1.6, f"depth 16:\n{r['layerwise'] / r['random']:.0f}x larger\nwith LL init",
            ha="right", va="bottom", fontsize=9, color=MUTED)
    ax.set_yscale("log")
    ax.set_xticks([1, 4, 8, 12, 16, 20])
    ax.set_xlabel("number of layers (10 qubits)")
    ax.set_title("(b) Our model: gradient variance vs depth\n"
                 "random init decays, LL init stays large")
    ax.legend(loc="lower left")
    fig.text(0.5, -0.04, "Gradient of the first rotation angle, 100 random initialisations on real MNIST "
             "inputs, shaded = 95% CI.  (a) global cost  <Z0 Z1 ... Zn-1>,  (b) local cost <Z0> "
             "(the cost used for training).", ha="center", fontsize=9, color=MUTED)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)

```

Figure 3: reads `bp_diagnostic.json`; panel (a) gradient variance vs number of qubits (global cost), panel (b) gradient variance vs depth for random and layer-wise starts, both on a logarithmic axis with shaded 95% CIs.

src/figures.py (lines 222-319)

```
# ------------------------------------------------------------ performance
def fig_curves(runs, path):
    fig, axes = plt.subplots(1, 3, figsize=(14, 4.4), sharey=True)
    for ax, s in zip(axes, SETTINGS):
        for m in METHODS:
            A = np.array([r["history"]["test_acc"] for r in runs[(s, m)]])
            it = np.array(runs[(s, m)][0]["history"]["iter"])
            mu, ci = A.mean(0), 1.96 * A.std(0, ddof=1) / np.sqrt(len(A))
            ax.plot(it, mu, color=COLOR[m], lw=2, label=METHODS[m])
            ax.fill_between(it, mu - ci, mu + ci, color=COLOR[m], alpha=0.18, lw=0)
        ax.axhline(C.ACC_TARGET, color=MUTED, lw=1, ls="--")
        ax.text(4, C.ACC_TARGET + 0.008, "90% target", ha="left", va="bottom", fontsize=9, color=MUTED)
        ax.set_title(SETTINGS[s].replace("\n", " "))
        ax.set_xlabel("training update")
        ax.set_xlim(0, 320)
        ax.set_xticks(range(0, 321, 80))
    axes[0].set_ylabel("test accuracy")
    axes[0].set_ylim(0.45, 1.0)
    axes[0].legend(loc="lower right")
    fig.suptitle("Test accuracy during training (mean of 30 seeds, shaded = 95% CI)", y=1.02)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def _grouped_bars(ax, rows, key, err=None, fmt="{:.2f}", extra=None):
    x = np.arange(len(SETTINGS))
    w = 0.36
    for i, m in enumerate(METHODS):
        vals = [_row(rows, s, m)[key] or 0 for s in SETTINGS]
        errs = [_row(rows, s, m)[err] for s in SETTINGS] if err else None
        xs = x + (i - 0.5) * w
        ax.bar(xs, vals, w * 0.92, color=COLOR[m], label=METHODS[m], yerr=errs,
               error_kw=dict(ecolor=INK, capsize=4, lw=1))
        for j, (xx, v) in enumerate(zip(xs, vals)):
            top = v + (errs[j] if errs else 0)
```

```
            lab = fmt.format(v) + (("\n" + extra(_row(rows, list(SETTINGS)[j], m))) if extra else "")
            ax.text(xx, top, lab, ha="center", va="bottom", fontsize=9, color=INK,
                    bbox=dict(fc="white", ec="none", pad=0.5), zorder=5)
    ax.set_xticks(x)
    ax.set_xticklabels(SETTINGS.values())
    ax.grid(axis="x", visible=False)


def fig_final_accuracy(rows, path):
    fig, ax = plt.subplots(figsize=(8, 4.8))
    _grouped_bars(ax, rows, "mean_test_acc", err="ci95_test_acc")
    ax.set_ylim(0, 1.08)
    ax.set_ylabel("final test accuracy")
    ax.set_title("Final test accuracy after 320 updates\n(mean of 30 seeds, error bar = 95% CI)")
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=2)
    fig.savefig(path)
    plt.close(fig)


def fig_success(rows, path):
    fig, ax = plt.subplots(figsize=(8, 4.8))
    _grouped_bars(ax, rows, "success_rate_pct", fmt="{:.0f}%",
                  extra=lambda r: f"({r['successful_runs']}/{r['runs']})")
    ax.set_ylim(0, 110)
    ax.set_ylabel("successful runs (%)")
    ax.set_title("Percentage of successful runs (30 seeds each)\n"
                 "success = final train loss < 0.347 and test accuracy ≥ 90%")
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=2)
    fig.savefig(path)
    plt.close(fig)


def fig_iterations(rows, path):
    fig, ax = plt.subplots(figsize=(8, 4.8))
    _grouped_bars(ax, rows, "median_iters_to_90pct", fmt="{:.0f}",
                  extra=lambda r: f"reached in\n{r['runs_reaching_90pct']}/{r['runs']} runs")
    ax.set_ylim(0, 420)
    ax.axhline(320, color=MUTED, lw=1, ls="--", zorder=1)
    ax.text(-0.45, 324, "training budget = 320", ha="left", va="bottom", fontsize=9, color=MUTED)
    ax.set_ylabel("training updates (median)")
    ax.set_title("Iterations to converge: updates until test accuracy first reaches 90%\n"
                 "(median over the runs that reached it; lower is faster)")
    ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=2)
    fig.savefig(path)
    plt.close(fig)


def make_all(log=print):
    C.FIGURES.mkdir(exist_ok=True)
    runs = load_all()
    rows = summarize(runs)
    log(f"wrote {write_summary(rows)}")
    F = C.FIGURES
    fig_circuit(F / "fig1_circuit.png")
    fig_training_schemes(F / "fig2_training_schemes.png")
    fig_barren_plateau(F / "fig3_barren_plateau.png")
    fig_curves(runs, F / "fig4_test_accuracy_curves.png")
    fig_final_accuracy(rows, F / "fig5_final_accuracy.png")
    fig_success(rows, F / "fig6_success_rate.png")
    fig_iterations(rows, F / "fig7_iterations_to_converge.png")
    log(f"figures written to {F}")
    return rows
```

Figures 4-7: `fig_curves` averages the test-accuracy history of 30 runs at every update and shades the 95% CI; `_grouped_bars` draws the Direct/LL bar pairs for the three settings with value labels; `fig_final_accuracy`, `fig_success` and `fig_iterations` use it for Figures 5-7. `make_all` writes the table and all 7 figures.

22.2 The notebook

`notebook/layerwise_vs_direct.ipynb` is the interactive deliverable. Its sections:

Section	What it does
Intro	the problem and objectives
1. Task and data	builds the data split and prints sizes and preprocessing
2. Model	shows Figure 1; runs the PennyLane circuit and the fast simulator on 4 test images and prints both outputs (identical, difference ~10^{-16}); prints the parameter count
3. Training schemes	shows Figure 2; prints both schedules; optional live training switch `RUN_DEMO = True` that retrains seed 0 of both methods in 1-2 minutes and compares with the saved result
4. Results	the results table, the significance tests, Figures 4-7
5. Barren-plateau diagnostic	Figure 3 and the per-parameter gradient sizes during training
6. Conclusion	the main conclusions

> **Say this to sir:** "figures.py computes the summary table and draws all 7 figures only from the saved result files. The notebook builds the model, verifies the simulator against PennyLane, shows both schedules, can retrain a seed live, and presents all results and the diagnostic."

23. Results: the numbers (Objective O3)

23.1 The main table

Each row is 30 runs (seeds 0-29); LL = Layer-wise. This is `results/summary.csv`.

Noise	Method	Mean test accuracy ± 95% CI	Worst / best run	Successful runs	Reached 90%	Median updates to 90%
Noiseless	**Direct**	**0.920 ± 0.009**	0.853 / 0.945	**25/30 (83%)**	30/30	**58**
Noiseless	LL	0.887 ± 0.011	0.785 / 0.935	3/30 (10%)	25/30	264
1000 shots	**Direct**	**0.914 ± 0.009**	0.858 / 0.955	**23/30 (77%)**	30/30	**65**
1000 shots	LL	0.794 ± 0.021	0.690 / 0.907	0/30 (0%)	4/30	285
500 shots	**Direct**	**0.906 ± 0.012**	0.812 / 0.943	**23/30 (77%)**	30/30	**71**
500 shots	LL	0.793 ± 0.018	0.713 / 0.900	0/30 (0%)	1/30	298

How to read one row, e.g. "Noiseless, Direct": averaged over 30 runs, the trained model classifies 92.0% of test images correctly; we are 95% sure the true average is between 91.1% and 92.9%; 25 of the 30 runs met the success rule; all 30 reached 90% test accuracy at some point, typically (median) after 58 updates.

23.2 Is the difference real? (significance)

Setting	Direct better than LL (same seed)	Wilcoxon p (accuracy)	Fisher p (success)
Noiseless	28 of 30 seeds	3.3×10^{-6}	1.1×10^{-8}
1000 shots	30 of 30 seeds	1.7×10^{-6}	1.7×10^{-10}
500 shots	30 of 30 seeds	1.7×10^{-6}	1.7×10^{-10}

All p-values are far below 0.05: the advantage of Direct is **not luck**.

23.3 No overfitting

Final *training* accuracy is almost equal to test accuracy (Direct noiseless: 92.4% train vs 92.0% test). So the models are not memorising the training images; with 322 parameters and 600 images this is expected.

23.4 Comparison with classical models

On the same 10 features, logistic regression reaches 91.7% and an SVM 95.8% test accuracy. Direct's 92.0% is at the level of logistic regression. (The goal of the project is to compare two training methods, not to beat classical ML.)

Say this to sir: "Direct reached 92% test accuracy noiseless and 91% with 1000 or 500 shots, with 77-83% successful runs and about 60-70 updates to reach 90%. Layer-wise reached 89% noiseless but only 79% with shots, with 10% or 0% success. Direct is better in 28 to 30 of 30 paired seeds, p below 10^-5."

24. Every graph explained

Colours everywhere: **orange = Direct, blue = Layer-wise**. Shaded bands and error bars are 95% confidence intervals over the 30 seeds.

24.1 Figure 1: the circuit

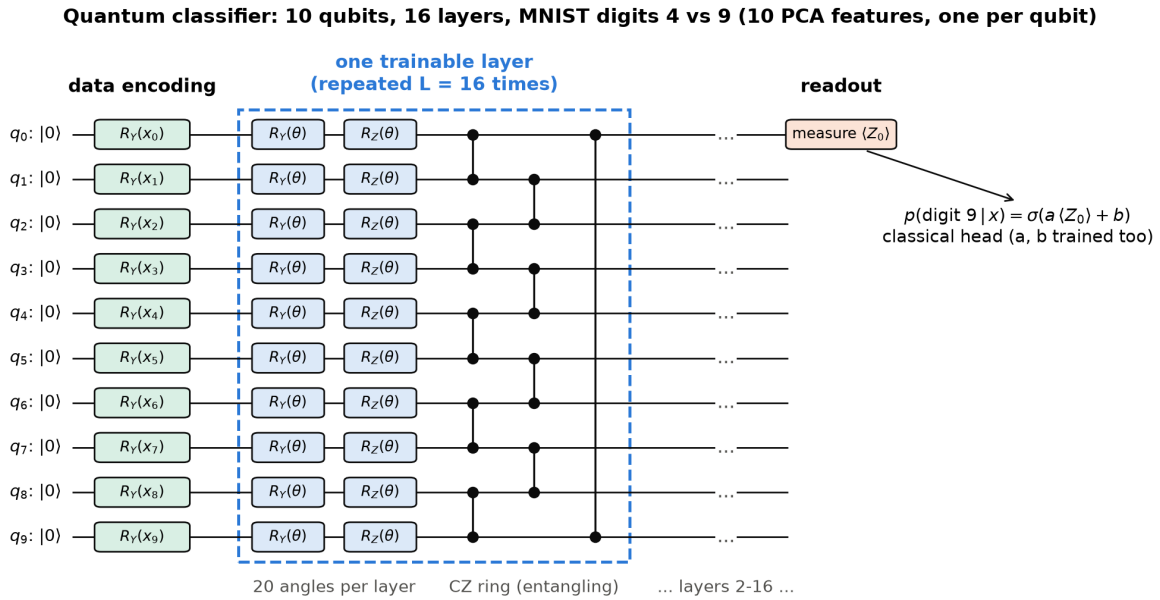


Figure 1.

- **What it shows:** our quantum classifier: 10 qubits (lines), data encoding RY(x_q) (green), one trainable layer (blue dashed box: RY(θ), RZ(θ) on every qubit, then the CZ ring), repeated 16 times, then measuring $\langle Z_0 \rangle$ and the classical sigmoid head.
- **How to read the CZ ring:** each vertical line with two dots is a CZ between neighbours; the long line connects qubit 9 back to qubit 0, closing the ring. CZ gates commute with each other, so drawing them in three columns does not change the circuit.
- **What to say:** "Each image's 10 PCA features are encoded as RY rotations; each of the 16 layers has 20 trainable angles and a CZ ring; we read out $\langle Z_0 \rangle$ and convert it to a probability with a sigmoid."

24.2 Figure 2: the two training schemes

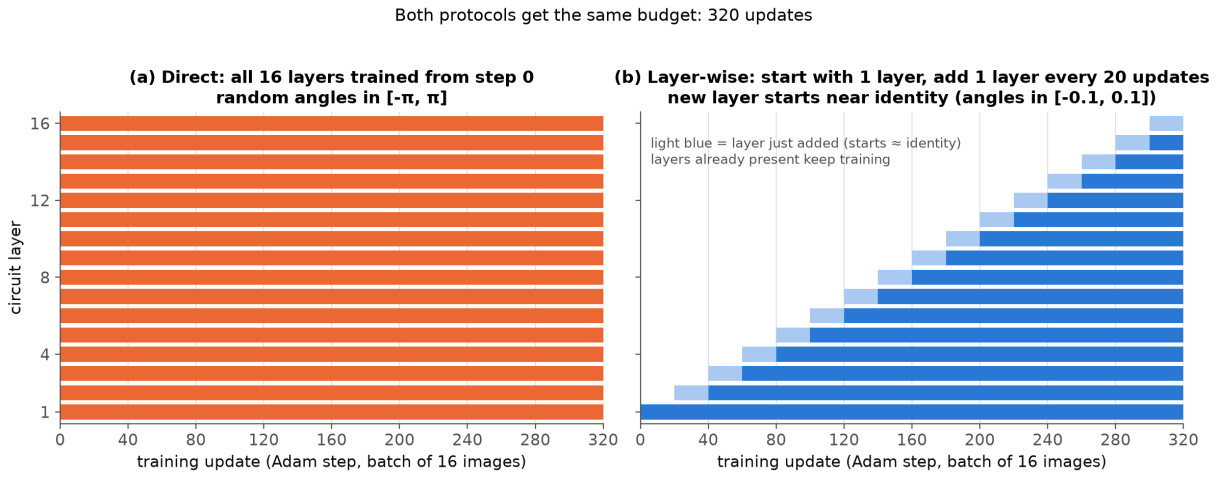
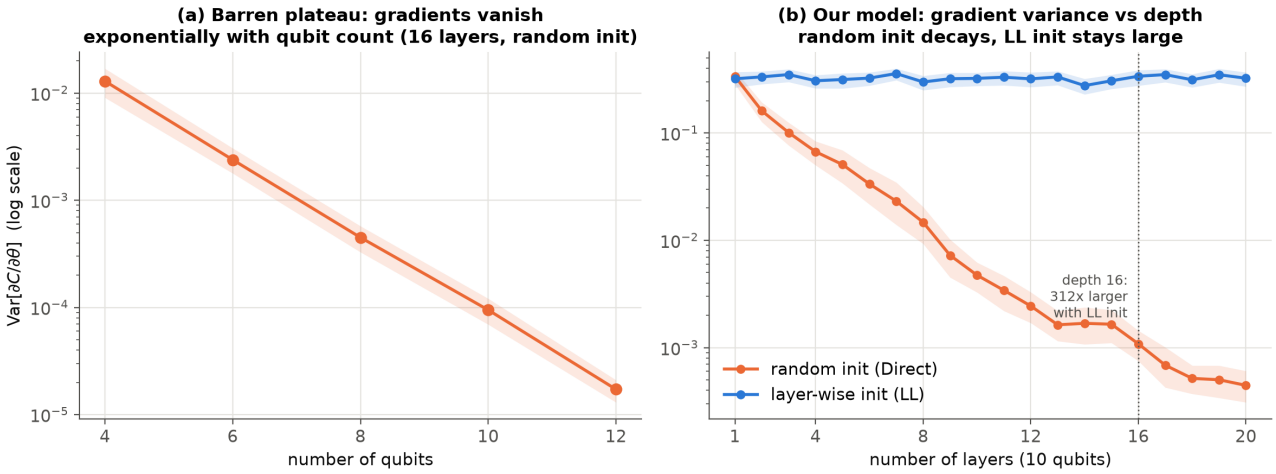


Figure 2.

- **What it shows:** x-axis = training update (0-320), y-axis = circuit layer (1-16). A bar means "this layer exists and is being trained at this time".
- **Left (Direct):** all 16 bars run the full width: every layer is trained for all 320 updates.
- **Right (Layer-wise):** a staircase: layer 1 from update 0, layer 2 from update 20, ..., layer 16 only from update 300. Light blue = the first 20 updates of a new layer, which starts near identity.
- **Key observation:** the area of the staircase is about half of the rectangle. LL spends most of its budget with a shallow circuit; the full 16-layer circuit is trained for only the last 20 updates.
- **What to say:** "Both get 320 updates, but LL grows one layer every 20 updates, so the full-depth circuit only exists in the last 20 updates."

24.3 Figure 3: the barren-plateau diagnostic



Gradient of the first rotation angle, 100 random initialisations on real MNIST inputs, shaded = 95% CI. (a) global cost <Z0 Z1 ... Zn-1>, (b) local cost <Z0> (the cost used for training).

Figure 3.

Panel (a): gradients vs number of qubits (global cost, 16 layers, random angles).

- y-axis: variance of the gradient over 100 random starts, on a **log scale** (each grid line is 10× smaller).
- The points fall on a straight line on the log scale: that means **exponential decay**: 1.3×10^{-2} at 4 qubits, 2.4×10^{-3} at 6, 4.5×10^{-4} at 8, 9.5×10^{-5} at 10, 1.7×10^{-5} at 12. Roughly 5× smaller for every 2 extra qubits. This is the textbook barren plateau.

Panel (b): gradients vs depth for our model (10 qubits, local cost $\langle Z_0 \rangle$).

- **Orange (random start, as in Direct):** the variance falls from 0.34 at 1 layer to 1.1×10^{-3} at 16 layers (about **300× smaller**) and keeps falling (4.4×10^{-4} at 20 layers).
- **Blue (layer-wise start):** stays at about 0.3 for *every* depth, because the extra near-identity layers barely change the circuit.
- **What it proves:** the mechanism behind layer-wise learning is real. Starting shallow / near identity keeps gradients about 300× larger at depth 16.
- **Important nuance:** for our local cost, the decay with qubit number at fixed depth is much milder (10 qubits: 1.1×10^{-3}, still measurable). That is why Direct can still train at our size (Chapter 26).
- **What to say:** "Panel (a) shows the barren plateau: gradient variance decays exponentially with qubits for a global cost. Panel (b) shows that in our model random initialisation shrinks the gradient variance 300 times by depth 16, while the layer-wise initialisation keeps it at the one-layer level."

24.4 Figure 4: test accuracy during training

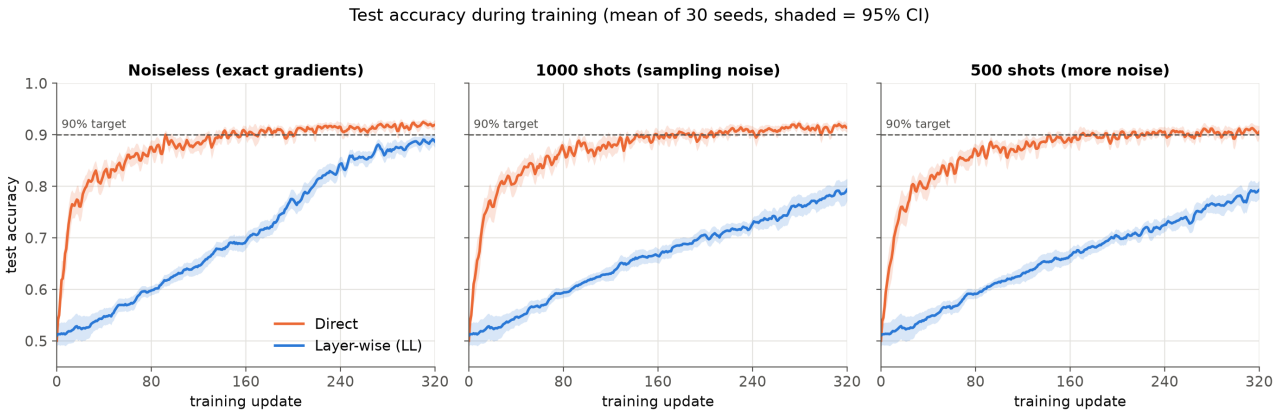


Figure 4.

- **What it shows:** the average test accuracy of the 30 runs after every update, for each noise setting. Dashed line: 90% target.
- **Direct (orange):** rises immediately: about 76% after 20 updates, 83% after 40, 88% after 80, and crosses 90% at around 100-160 updates on average, ending at 91-92%. Almost the same curve in all three panels: **shot noise barely affects Direct**.
- **Layer-wise (blue), noiseless:** almost flat at first: 52.5% after 20 updates (1 layer), 55% after 40, 60% after 80, 69% after 160; then it climbs faster as more layers are added (84% at 240) and ends at 88.7%, close to but below Direct.
- **Layer-wise with shots:** the same slow start, but it climbs much more slowly later (73% at 240) and ends at only 79%.
- **Why Direct starts at exactly 50%:** with 16 random layers the output $\langle Z_0 \rangle$ is almost exactly 0 for every image (the concentration effect behind barren plateaus), so every prediction is $p \approx 0.5$. But the gradient is still large enough to escape quickly.
- **What to say:** "Direct converges within about 100 updates and is unaffected by shot noise. LL learns slowly while the circuit is shallow, almost catches up without noise, but with shots it stays around 79%."

24.5 Figure 5: final test accuracy and noise sensitivity

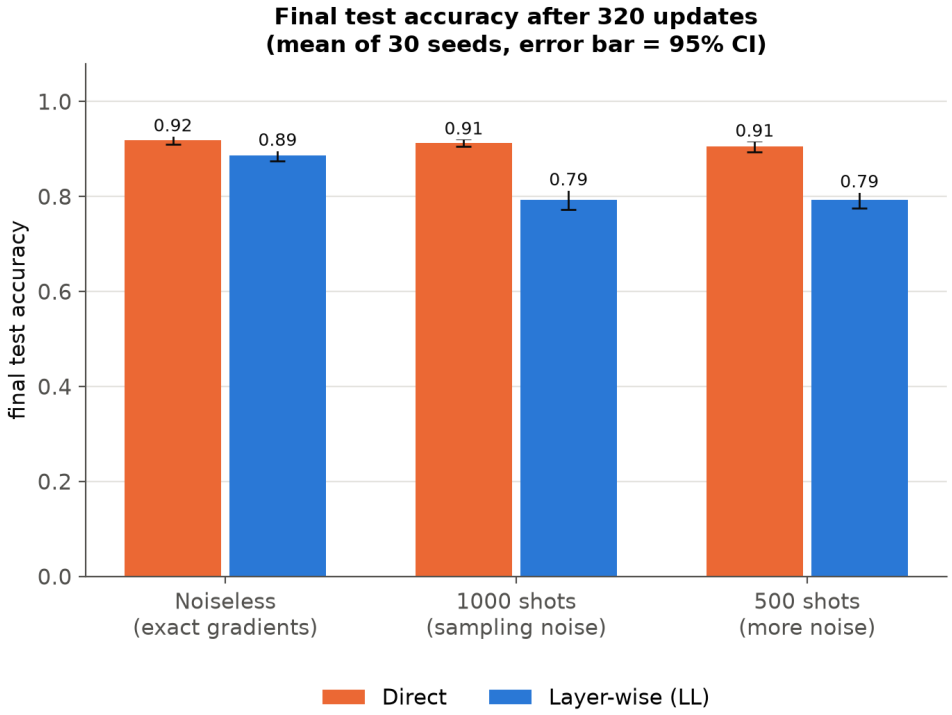


Figure 5.

- **What it shows:** the final test accuracy after 320 updates (bar = mean of 30 runs, black whisker = 95% CI) for each noise level.
- **Sensitivity to sampling noise:** read each colour from left to right. Direct: 0.92 → 0.91 → 0.91 (loses about 1.4 points). LL: 0.89 → 0.79 → 0.79 (loses about 9 points).
- The whiskers of Direct and LL never overlap: the difference is clear.
- **What to say:** "Under sampling noise Direct loses about 1 percentage point while LL loses about 9, so here LL was more noise-sensitive."

24.6 Figure 6: percentage of successful runs

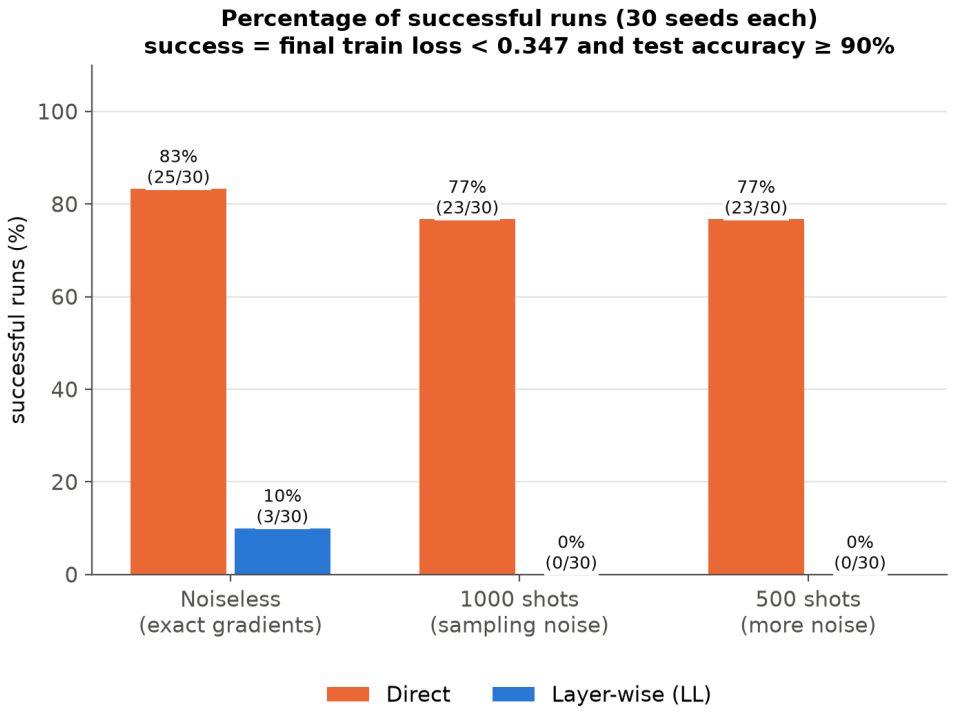


Figure 6.

- **What it shows:** how many of the 30 runs met the success rule (final training loss < 0.347 and test accuracy ≥ 90%).
- Direct: 83%, 77%, 77%. LL: 10%, 0%, 0%.
- Why LL's success is so low even noiseless although its accuracy is 88.7%: the success rule also needs a low training loss, and LL's final loss (0.37 on average) is just above the 0.347 threshold because the full circuit only trained for 20 updates.
- **What to say:** "Direct succeeded in about 80% of runs in all settings; LL in 10% noiseless and none with shots."

24.7 Figure 7: iterations to converge

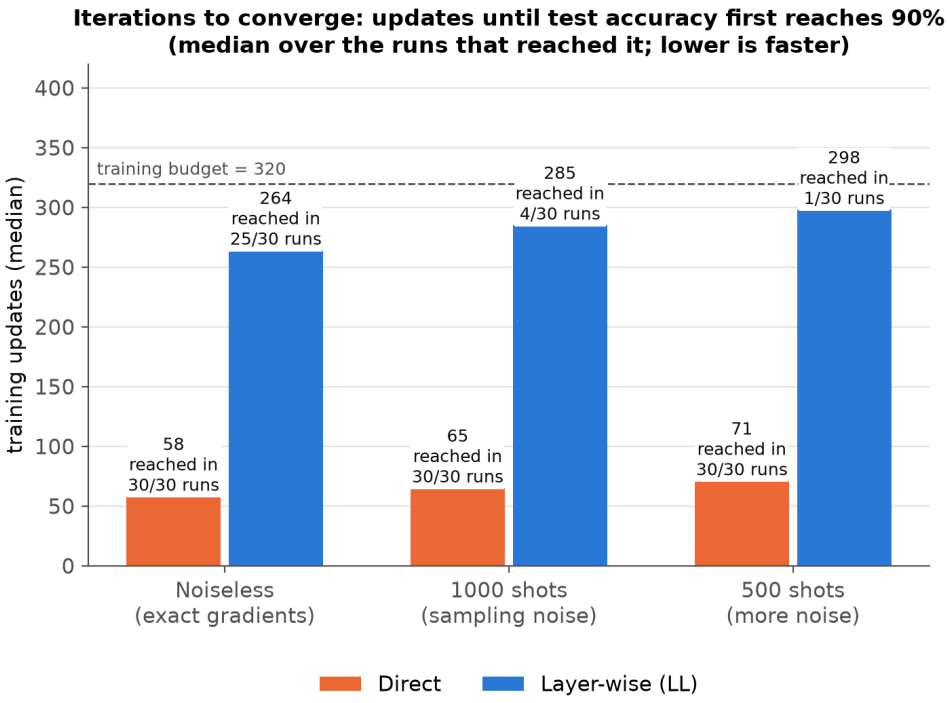


Figure 7.

- **What it shows:** the number of updates until test accuracy first reached 90% (median over the runs that reached it). The label says how many of the 30 runs reached 90% at all.
- Direct: 58, 65, 71 updates, and **every** run reached 90%.
- LL: 264, 285, 298 updates, and only 25, 4 and 1 of 30 runs reached it. The dashed line is the budget (320).
- Note: for LL the median is over the few runs that did reach 90%, so it *flatters* LL; runs that never reached it are not counted.
- **What to say:** "Direct reaches 90% in about 60-70 updates in every run. LL needs over 260 updates and, with shots, almost never gets there."

Key idea: All three O3 measures point the same way: Direct had higher final accuracy, converged about 4-5× faster, succeeded far more often, and was much less sensitive to shot noise.

25. A deeper look inside training

25.1 The training loss over time (average of 30 runs)

Update	0	20	40	80	100	160	200	240	280	320
Direct, noiseless	0.693	0.623	0.546	0.433	0.398	0.318	0.298	0.264	0.251	0.234
LL, noiseless	0.753	0.691	0.682	0.670	0.660	0.616	0.560	0.483	0.421	0.371
Direct, 500 shots	0.693	0.635	0.569	0.452	0.416	0.337	0.316	0.286	0.270	0.262
LL, 500 shots	0.753	0.690	0.683	0.673	0.665	0.637	0.612	0.585	0.549	0.508

- LL is **stuck near 0.69** (coin flip) for about the first 100 updates (depths 1-5). This is *not* a barren plateau: its gradients are not vanishing; the shallow circuit simply cannot do better (see the light cone, Chapter 26).
- LL starts at 0.753, above 0.693: a one-layer circuit with random angles gives confident but random predictions, which costs more than guessing 50/50.

25.2 Gradient size per parameter during training (noiseless)

Update	20	40	100	200
Direct: mean abs. gradient per angle	0.0041	0.0068	0.0136	0.0184
LL: mean abs. gradient per angle	0.0023	0.0018	0.0027	0.0057

Surprising at first: although LL's *starting point* has large gradients (Figure 3), during training LL's gradients per angle are **2-5 times smaller** than Direct's. Reason: after a few updates the shallow circuit has learned all it can, so there is little left to improve (small slope), while Direct, with all features in view, keeps finding useful directions.

> **Say this to sir:** "LL's training loss stays at the coin-flip level for about 100 updates, not because of a barren plateau but because the shallow circuit has too little capacity. During training its gradients per parameter are 2 to 5 times smaller than Direct's."

26. Analysis: why Layer-wise learning did not win here

The brief expected LL to win for deep circuits. Three reasons, supported by our data, explain why it did not at our scale.

26.1 Reason 1: Direct is not on a barren plateau at 10 qubits

- LL is a cure for barren plateaus. It can only help if the patient is ill.
- With a *local* cost ($\langle Z_0 \rangle$) and 10 qubits, Direct's gradient variance at 16 layers is about 10^{-3}: small, but clearly usable. The exponential collapse in Figure 3(a) happens with a *global* cost or many more qubits.
- Evidence from training: Direct's accuracy rises from the very first updates (Figure 4); there is no flat start.
- Evidence from noise: Direct with 500 shots (0.906) is almost as good as noiseless (0.920). If its gradients were drowned in shot noise, accuracy would collapse. It did not.

26.2 Reason 2: shallow circuits cannot see the whole image (light cone)

- Because we measure only qubit 0 and the CZ gates connect neighbours, a depth-d circuit only depends on **2d - 1** of the 10 features (Chapter 9.3). Depth 1 sees only x_0, depth 3 sees 5 features; all 10 only from depth 6.
- So for the first 100 updates (depths 1-5) LL's circuit is missing important information and cannot separate 4 from 9 well; its loss stays near 0.69 (Chapter 25).
- With an equal budget of 320 updates, LL spends most updates on these weak circuits, and the full 16-layer circuit trains for only 20 updates. Direct uses the full circuit for all 320.

26.3 Reason 3: smaller gradients suffer more from shot noise

- Shot noise adds the same random error (about 0.03-0.045 per circuit value) to both methods.
- During training LL's gradients per angle are 2-5× smaller than Direct's (Chapter 25.2), so the noise is relatively 2-5× larger for LL: a worse **signal-to-noise ratio**. Its updates become partly random, so it learns even more slowly: 79% instead of 89%.
- Direct's larger gradients keep a good signal-to-noise ratio, so it is almost unaffected.

26.4 When would layer-wise learning win?

LL should help where Direct really stalls:

- many more qubits (the gradient variance shrinks exponentially with qubit count);
- a global cost function;
- much deeper circuits (e.g. 50-100 layers);
- or LL with more updates per stage (an earlier version of this project gave LL 20 extra updates per depth on top of the 320 and it then *matched* Direct, 0.92 accuracy).

These large settings cannot be simulated on a laptop (the state of n qubits needs 2^n numbers), which is a fundamental limitation of all simulation studies.

> **Say this to sir:** "Three reasons. First, at 10 qubits with a local cost Direct is not on a barren plateau: its gradients are small but usable, and shot noise barely hurt it. Second, a shallow circuit measuring qubit 0 only sees 2d−1 features, so LL spends most of its equal budget on circuits too weak to classify, and the full circuit trains only 20 updates. Third, LL's gradients during training are smaller, so shot noise hurts it more. LL should help at larger scales where direct training actually stalls."

27. Conclusion

1. We built a 10-qubit, 16-layer variational quantum classifier for real MNIST digits 4 vs 9 (O1).
2. We implemented Direct training and Layer-wise training with the same budget, data, learning rate and paired seeds (O2).
3. Over 180 runs we compared final accuracy, iterations to converge, success rate and sensitivity to 1000- and 500-shot sampling noise (O3).
4. The barren-plateau diagnostic confirms the theory: random initialisation shrinks gradients exponentially with qubits and by 300× with depth, and the layer-wise initialisation keeps them large.
5. Nevertheless, at this scale **Direct training was better on every measure**, with high statistical significance. The expected advantage of LL was **not** reproduced here, and we explained why: no plateau for Direct at 10 qubits with a local cost, the light-cone limit of shallow circuits, and LL's worse signal-to-noise ratio under shots.
6. Lesson: a barren-plateau mitigation only pays off when there is a barren plateau. Whether a strategy helps depends on the circuit size, cost function and budget.

28. Limitations and future work

Limitation	Why it matters	Future work
Only 10 qubits	barren plateaus are mild here	larger qubit counts on a GPU or real hardware
One circuit design (local cost, CZ ring)	other designs may behave differently	global cost, other entanglers
One LL variant (joint, 20 updates per stage)	the paper's frozen variant and longer stages may do better	more stage lengths, frozen LL
Only sampling noise	real devices also have gate errors and decoherence	depolarising-noise models or real hardware
One digit pair	results may depend on the task	other pairs, more classes
Budget matched by updates	LL uses fewer circuit executions with shots	compare at matched circuit executions

Say this to sir: "Limitations: only 10 qubits, one circuit design and one LL variant, only sampling noise, one digit pair. Future work: larger or global-cost circuits, frozen LL and longer stages, and hardware noise."

29. How we know everything is correct

Check	Result
Fast simulator vs PennyLane: output values	identical up to 3×10^{-16} (shown live in the notebook)
Fast simulator vs PennyLane: gradients (PennyLane's own parameter-shift)	agree up to 4×10^{-16}
Our parameter-shift values vs direct evaluation with shifted angles	agree up to 10^{-15}
Shot sampling	unbiased; variance 1.79×10^{-3} vs theory 1.82×10^{-3} (500 shots)
Average of 300 noisy shot gradients vs exact gradient	agree (difference 0.0007 on gradients of size 0.11)
Zero-angle layer appended	output change exactly 0.0
Light cone	exactly 2d - 1 features influence the output at depth d
All 180 saved runs	settings, data split fingerprint, layer schedule, success flags and iterations recomputed: 0 problems
Re-evaluating all 180 saved final models on the test set	every test accuracy reproduced exactly
Retraining seed 0 of both methods from scratch	same test accuracy (0.930 and 0.9125); loss equal to 10^{-7}
Significance	Wilcoxon and Fisher tests, $p < 10^{-5}$

Say this to sir: "We verified the simulator against PennyLane to machine precision, checked the shot sampler statistically, re-checked all 180 result files and re-evaluated every saved model, and retraining a seed reproduces the saved result."

30. Running the project and demo for sir

30.1 One-time setup (Windows)

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

(The `.venv` folder already exists on the project laptop, so this is only needed on a new computer.)

30.2 Commands

Command	Time	What happens
`.\.venv\Scripts\python.exe run_all.py figures`	~1 min	rebuilds `results/summary.csv` and all 7 figures from the saved results
`.\.venv\Scripts\python.exe run_all.py train`	~5 h	runs all 180 trainings (finished runs are skipped, so with the saved results it does nothing)
`.\.venv\Scripts\python.exe run_all.py diagnostic`	~1 min	barren-plateau diagnostic (skipped if the file exists)

30.3 Showing the notebook

1. Open VS Code (or Jupyter) and open `notebook/layerwise_vs_direct.ipynb`.
2. Select the `.venv` Python as the kernel.
3. Run all cells (about 30 seconds). It prints the data sizes, the PennyLane vs simulator check, both schedules, the results table, significance tests, and shows all figures.
4. **Live demo:** in Section 3 set `RUN_DEMO = True` and run that cell: it trains seed 0 of both methods from scratch (1-2 minutes) and prints the same accuracies as the saved results (Direct 0.9300, LL 0.9125).

30.4 Suggested 10-minute presentation order

1. Problem: barren plateaus (Chapter 5, landscape picture).
2. Idea of layer-wise learning (Chapter 6).
3. Task and data (Chapter 8 picture).
4. The circuit (Figure 1).
5. The two schemes (Figure 2).
6. Barren-plateau diagnostic (Figure 3): "the mechanism is real".
7. Training curves (Figure 4), then Figures 5-7.
8. Why Direct won (Chapter 26: three reasons).
9. Conclusion and limitations.
10. Show the notebook, optionally the live demo.

31. Viva questions and answers

31.1 Basics

Q1. What is a variational quantum circuit? A quantum circuit with trainable rotation angles. The quantum device evaluates it, a classical optimiser updates the angles to minimise a loss: a hybrid quantum-classical algorithm.

Q2. What is a barren plateau? A region where the loss landscape is almost flat: the gradient variance shrinks exponentially with the number of qubits, so gradients are effectively zero and the optimiser cannot find a direction, especially with shot noise.

Q3. Why do barren plateaus happen? Deep circuits with random angles produce almost random quantum states; measured averages of random states are extremely concentrated around their mean, so the output hardly changes when an angle changes.

Q4. What is layer-wise learning? Training a shallow circuit first and adding layers one at a time, each starting near identity, so the circuit is always effectively shallow with large gradients (Skolik et al. 2021).

Q5. What is the parameter-shift rule? An exact way to get the gradient on hardware: $df/d\theta = [f(\theta+\pi/2) - f(\theta-\pi/2)]/2$ for rotation gates. It needs 2 extra circuit evaluations per parameter.

Q6. What is a shot? One run of the circuit followed by measurement, giving a single 0 or 1 per qubit. Expectation values are averages over many shots; with S shots the error is about $1/\sqrt{S}$.

Q7. What is $\langle Z_0 \rangle$? The expectation value of Pauli-Z on qubit 0: P(0) - P(1), between -1 and +1. It is our model's raw output.

Q8. What is entanglement and where is it in your circuit? Correlation between qubits that cannot be described qubit by qubit; created by the CZ ring in every layer.

31.2 Our design

Q9. Why MNIST 4 vs 9? Real data, and one of the most confusable digit pairs, so the task is not trivial and differences between methods can show.

Q10. Why 10 qubits and PCA to 10 features? One feature per qubit with angle encoding; 10 qubits is large enough to see gradient decay and still simulable for 180 runs. PCA keeps 64.6% of the image variance.

Q11. Why are PCA and scaling fitted on training data only? Otherwise information about the test images would leak into training and the test score would be over-optimistic.

Q12. Why 16 layers? Deep enough that random initialisation shrinks gradients about 300× (a clear barren-plateau effect in our diagnostic), so it is a "deep circuit" in the sense of the brief.

Q13. Why a local cost $\langle Z_0 \rangle$ and not a global one? Global costs have barren plateaus even for shallow circuits, so both methods would fail; local costs are the standard trainable choice (Cerezo et al. 2021).

Q14. Why CZ gates? CZ commutes with Z measurements, so a new layer with zero angles leaves the output exactly unchanged: exactly what "add layers near identity" needs. CZ and CNOT have the same entangling power.

Q15. Why do new LL layers start in ±0.1 and not random ±π? So they are almost the identity and do not destroy what was learned; random ±π layers would scramble the state (the barren-plateau starting point).

Q16. Why do all layers keep training in your LL (joint) instead of freezing? It matches the brief and is simpler; it gives LL more freedom. An earlier frozen variant was not better.

Q17. What is the classical head $\sigma(a\langle Z_0\rangle+b)$? Two trainable classical numbers that scale and shift the quantum output before the sigmoid, so it can produce confident probabilities.

Q18. Why Adam with learning rate 0.03? Adam is the standard optimiser and was named in the brief; 0.03 was chosen on the validation set from 0.01, 0.03, 0.1 before the main experiment, the same for both methods.

Q19. Why 30 seeds? Results vary with random initialisation and batch order; 30 seeds give reliable averages, confidence intervals and success rates, and allow paired statistical tests.

Q20. What does "paired" mean here? For the same seed both methods start with the same first layer and see the same sequence of mini-batches, so differences come from the method, not luck.

Q21. Is the comparison fair? Yes: same data, circuit, learning rate, batch size and number of updates (320). With shots LL even used fewer circuit executions (1.75 M vs 3.28 M).

Q22. How did you simulate shot noise? Every circuit value is replaced by 2k/S - 1 where k ~ Binomial(S, (1+f)/2): exactly a device's measurement statistics; gradients use the parameter-shift rule on these noisy values.

Q23. Why a custom simulator instead of PennyLane? Speed: 180 runs with parameter shifts and shots would take weeks in PennyLane. Our PyTorch simulator implements the same circuit and is verified against PennyLane to about 10^{-16}.

31.3 Results and analysis

Q24. What are your main results? Direct: 92% / 91% / 91% test accuracy (noiseless / 1000 / 500 shots), 77-83% success, ~60-70 updates to 90%. LL: 89% / 79% / 79%, 0-10% success, 264+ updates.

Q25. So does layer-wise learning work? Its mechanism works (Figure 3: gradients stay large), but at our scale it did not improve training, because Direct was not on a plateau.

Q26. Why did LL lose? (1) No barren plateau for Direct at 10 qubits with a local cost; (2) shallow circuits see only 2d - 1 features, so LL wastes most of its budget on weak circuits and trains the full circuit only 20 updates; (3) LL's smaller gradients have a worse signal-to-noise ratio under shots.

Q27. Your result contradicts the expected outcome. Is something wrong? No: we checked everything (Chapter 29), and the result is statistically significant. The expected advantage requires a regime where Direct stalls (more qubits, global cost, deeper circuits) that we could not simulate. Reporting this honestly is correct science.

Q28. What is the light cone? The set of input qubits that can influence the measured qubit. With nearest-neighbour CZ and readout on qubit 0, depth d covers 2d - 1 qubits.

Q29. Why does Direct start at exactly 50% accuracy? A 16-layer random circuit outputs $\langle Z_0\rangle \approx 0$ for every image (concentration), so $p \approx 0.5$ for all: this is the barren-plateau tendency, but the gradients are still large enough to leave quickly.

Q30. Why is LL's success rate 10% though its accuracy is 89%? Success also requires the training loss below 0.347; LL ends at 0.37 on average because the full circuit trained only 20 updates.

Q31. What do the confidence intervals mean? If we repeated the 30-seed experiment many times, 95% of such intervals would contain the true average. Non-overlapping intervals indicate a clear difference.

Q32. What does the p-value mean? The probability of seeing such a large difference if both methods were really equal. Ours are below 0.00001.

Q33. What is gradient variance and why log scale? The spread of gradient values over random starts, i.e. their typical size squared. It changes over many orders of magnitude, so a log axis shows exponential decay as a straight line.

Q34. How would you make LL win? More qubits or a global cost (where Direct stalls), or give LL extra updates per stage: in an earlier run with 20 extra updates per depth, LL matched Direct at 92%.

Q35. Is the model overfitting? No: training accuracy (92.4%) ≈ test accuracy (92.0%).

Q36. How good is 92% compared with classical ML? On the same 10 features logistic regression gets 91.7% and an SVM 95.8%. Our aim was comparing training methods, not beating classical ML.

Q37. How do you know the code is correct? Simulator equals PennyLane to machine precision; shot sampler statistically checked; all 180 files re-checked; every saved model re-evaluated; a retrained seed reproduces the saved result.

Q38. What would happen on a real quantum computer? Additional gate errors and decoherence; both methods would degrade, and deeper circuits more. Our study models sampling noise only.

Q39. How many circuit executions does one Direct update need with shots? 16 images × (1 + 2 × 320) = 10,256 circuits, each with 500 or 1000 shots.

Q40. What was the hardest part? Making 180 shot-based trainings feasible: we wrote and verified a fast simulator with an exact parameter-shift implementation.

32. Glossary

Term	Meaning
Accuracy	fraction of images classified correctly
Adam	popular improved gradient-descent optimiser with momentum and per-parameter step sizes
Ansatz	the chosen shape of the parameterised circuit
Angle encoding	putting data into a circuit as rotation angles, RY(x_q)
Barren plateau	almost flat loss landscape; gradient variance vanishes exponentially with qubits
Batch / mini-batch	the 16 images used for one update
BCE (binary cross-entropy)	the loss for yes/no classification; 0.693 = guessing
Bloch sphere	picture of a qubit state as an arrow on a sphere
Bootstrap	estimating uncertainty by resampling the data many times
Confidence interval (95%)	range that contains the true mean with 95% confidence
Cost / loss	number measuring how wrong the model is (lower = better)
CZ gate	two-qubit controlled-Z gate; creates entanglement
Depth	number of layers in the circuit
Direct training	training the full-depth circuit from random angles
Entanglement	quantum correlation between qubits
Epoch	one pass through the whole training set (37.5 updates here)
Expectation value ⟨Z⟩	average measurement result, P(0) - P(1)
Fisher's exact test	significance test for comparing two success counts
Global cost	measuring Z on all qubits together
Gradient	slope of the loss with respect to each parameter
Hybrid quantum-classical	quantum device + classical computer working in a loop
Identity	an operation that changes nothing
Layer	one repeating block of gates (RY, RZ, CZ ring)
Layer-wise learning (LL)	grow the circuit layer by layer during training
Learning rate	step size of the optimiser (0.03)
Light cone	the inputs that can influence a measured qubit
Local cost	measuring a single qubit ($\langle Z_0 \rangle$)
MNIST	dataset of 70,000 handwritten digit images
NISQ	noisy intermediate-scale quantum (today's quantum computers)
Parameter-shift rule	exact gradient from two shifted circuit runs
PCA	principal component analysis: compresses data to the most important directions
PennyLane	Python library for quantum machine learning
p-value	probability of a difference this large by chance
Qubit	quantum bit

Term	Meaning
QNode	PennyLane's differentiable quantum circuit object
RY, RZ	single-qubit rotation gates about the Y and Z axes
Seed	number that fixes the randomness so a run can be repeated
Shot	one execution and measurement of the circuit
Shot noise	random error from estimating averages with finitely many shots
Sigmoid σ	function squashing any number into 0..1
Signal-to-noise ratio	gradient size divided by noise size
Success (our definition)	final training loss < 0.3466 and test accuracy ≥ 90%
Superposition	a qubit being in a combination of 0 and 1
Test set	images never used in training, for the final honest score
Update / iteration	one optimiser step
Validation set	images used to choose settings before the main experiment
Variance	average squared spread around the mean
Wilcoxon test	paired significance test that does not assume a bell curve

References

1. A. Skolik, J. R. McClean, M. Mohseni, P. van der Smagt, M. Leib, "Layerwise learning for quantum neural networks", *Quantum Machine Intelligence* 3, 5 (2021).
2. J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, H. Neven, "Barren plateaus in quantum neural network training landscapes", *Nature Communications* 9, 4812 (2018).
3. M. Cerezo, A. Sone, T. Volkoff, L. Cincio, P. J. Coles, "Cost function dependent barren plateaus in shallow parametrized quantum circuits", *Nature Communications* 12, 1791 (2021).
4. M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, N. Killoran, "Evaluating analytic gradients on quantum hardware" (parameter-shift rule), *Physical Review A* 99, 032331 (2019).
5. Y. LeCun, C. Cortes, C. J. C. Burges, "The MNIST database of handwritten digits".
6. V. Bergholm et al., "PennyLane: Automatic differentiation of hybrid quantum-classical computations", arXiv:1811.04968 (2018).